

ZimaBlue: Evolving Generalizable World Action Models through Scalable Video Pre-training

Joy Future Academy¹

Abstract

Robotic manipulation faces a fundamental scaling challenge: robust generalization demands broad physical experience, yet action-labeled robot trajectories are expensive to collect and inherently limited in diversity. Egocentric videos offer a far more scalable source of embodied experience, capturing object interactions, contact dynamics, tool use, and long-horizon behaviors across diverse environments. The central challenge is how to convert this abundant but action-free experience into effective robot control. We introduce **ZimaBlue**, a scalable framework for learning generalizable World Action Models (WAMs) from large-scale video. ZimaBlue follows a three-stage training curriculum: it first performs causal embodied video pre-training on large-scale human and robot egocentric videos, then grounds the learned visual dynamics in heterogeneous robot trajectories through video-action mid-training with a unified action representation, and finally specializes the model to a target robot for deployment. To make generative WAMs practical for real-time control, ZimaBlue further adopts an asynchronous Slow-Fast dual-system architecture, where a high-capacity Slow world model provides generalizable spatiotemporal representations and a lightweight Fast branch enables **30 Hz action prediction** on NVIDIA RTX 4090. On real-robot zero-shot evaluations, scaling from target-robot data alone to over **120,000 hours of embodied video improves success from 36.1% to 77.8%**. ZimaBlue further delivers strong performance across multiple benchmarks, with particularly pronounced gains on unseen tasks.

Project: <https://zimablue-wam.github.io/>

Code: <https://github.com/ZimaBlue-WAM/ZimaBlue>

1 Introduction

A central obstacle in general-purpose robotics is not the absence of capable policy architectures, but the lack of a scalable source of embodied experience. A useful robot must follow open-ended language instructions, and execute new skills under changes in scene layout, viewpoint, dynamics, and embodiment. Recent vision-language-action (VLA) models [1–8] have made important progress on the first requirement. By extending pretrained vision-language models to motor control, they inherit strong semantic priors, can ground instructions in diverse objects and scenes, and usually support efficient feed-forward action prediction. Yet their generalization is still bounded by the action-labeled robot data used for policy learning. In practice, a VLA may understand what an instruction means while still lacking the spatial, dynamic, and motor knowledge required to perform a new manipulation skill [9]. This gap reflects a deeper bottleneck: reactive policies rely on action-supervised robot data not only to learn control, but also to acquire perceptual and physical knowledge that could otherwise be learned from much larger video corpora.

World Action Models (WAMs) [9–15] provide a different way to organize the learning problem, predicting how the world will evolve and how the robot should act within that evolving world. This joint video-action objective explicitly links visual understanding to the physical consequences of action. A misaligned grasp, a slipping object, or an occlusion during manipulation is not just visual variation; it is part of the causal structure that the model must explain. This is why recent WAMs such as DreamZero [9] and LingBot-VA [11] are appealing for robot learning: they make control depend less on memorizing labeled trajectories and more on understanding which visual changes are reachable. In this case, video becomes a scalable substrate for learning physical and causal priors before dense robot actions are available.

This distinction matters because the most scalable embodied data today is not robot trajectories, but video. Real-robot demonstrations remain expensive, even with teleoperation and UMI systems [5, 16–18]; simulation provides useful coverage but suffers from a persistent sim-to-real gap and still misses much of the visual, material, and contact diversity

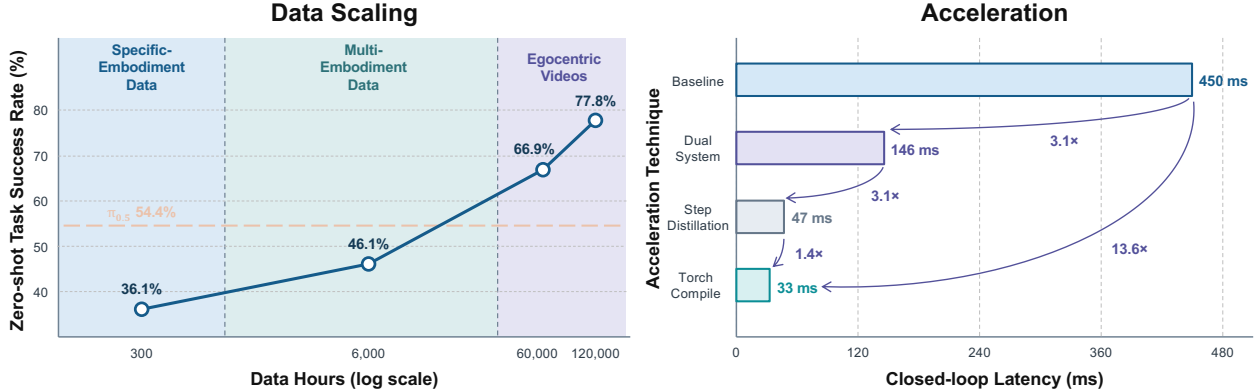


Figure 1: Performance scaling with expanded data and acceleration. **Left:** Zero-shot task success rate monotonically increases as the training data scales up from specific-embodiment data (36.1% at 300 hours) to multi-embodiment datasets (46.1% at 6,000 hours), and improves up to 77.8% when further incorporating large-scale egocentric video data (120,000 hours). **Right:** Progressive acceleration techniques cut closed-loop latency from 450 ms to 33 ms on NVIDIA RTX 4090, achieving a 13.6 $\times$ overall speedup.

of the real world [3, 19, 20]. By contrast, first-person human videos are abundant and easy to expand [21–25]. They contain rich evidence about object affordances, tool use, contact events, failure recovery, and long-horizon task structure, but most of them do not include action labels. This makes them hard to use in standard VLA training. A WAM can exploit them more naturally: it can first learn causal visual dynamics from unlabeled video, then align those dynamics with robot states and actions using a smaller amount of action-labeled data. We investigate this scaling strategy, with controlled comparisons across target-robot data, cross-embodiment robot data, and large-scale egocentric video pre-training. This separation is the key reason we choose the WAM paradigm for scaling embodied generalization.

However, simply initializing a WAM from an off-the-shelf video generator is not enough. Several recent video-action models [9, 11] inherit powerful spatiotemporal priors from large video diffusion backbones [26], which were originally designed for generic video synthesis rather than robot control. This creates several mismatches. First, generic video models are usually optimized for visually plausible reconstruction under descriptive prompts, whereas a robot needs instruction-conditioned prediction that is action-relevant, and grounded in the consequences of intervention. Second, many video backbones either attend across an entire clip or generate long chunks at once, whereas a deployed controller receives observations sequentially and can condition only on what has already happened. Finally, web video contains animation, visual effects, edited transitions, and other non-physical content that may attenuate the priors needed for contact-rich manipulation. These limitations suggest that WAMs should be specialized through embodied video pre-training, rather than retrofitted from generic video generation alone.

We present ZimaBlue, a WAM training framework built around this principle. The core hypothesis is simple: scaling causal embodied video pre-training improves downstream robotic generalization. ZimaBlue follows a three-stage recipe. **First**, we perform causal video pre-training on a large corpus of first-person and robot videos, scaling up to over 120,000 hours in our study. The model predicts future visual states from past observations and language instruction, encouraging it to learn semantic, temporal, and physical regularities before it sees target-robot actions. **Second**, we use multi-embodiment robot trajectories to connect the learned video dynamics to executable control. To make data from different embodiments useful in a single model, we introduce a unified action representation that standardizes robot states and actions across heterogeneous platforms. This stage turns broad video priors into action-aware representations, preventing the model from overfitting to a single robot. **Third**, we perform specific-embodiment post-training, which calibrates the model to the deployment embodiment and its action space. The resulting pipeline uses each data source for its natural role: abundant world knowledge from video, action grounding from cross-embodiment robot data, and precise control from target-robot demonstrations.

Our main evaluation asks whether this scaling path improves generalization, not merely video prediction quality or in-distribution imitation. We construct a suite of zero-shot manipulation tasks that are absent from the training data. As illustrated in Figure 1, the results show an appealing and consistent scaling trend. With only target-robot post-training, the model reaches 36.1% success. Adding cross-embodiment video-action mid-training improves success to 46.1%, showing that heterogeneous robot data provides transferable action grounding. Adding large-scale video pre-training further improves performance to 66.9% with 60,000 hours of video and 77.8% with 120,000 hours. These gains suggest that video pre-training plays an important role in improving generalization beyond the demonstrated tasks. In this sense, first-person video provides an effective and economical scaling axis for embodied intelligence.

We further evaluate ZimaBlue on standard benchmarks. On LIBERO-Plus [27], we outperform all WAM-based methods. On RoboTwin 2.0 [28], it achieves competitive results across diverse simulated manipulation tasks. On the challenging

RoboCasa365 [29] benchmark, ZimaBlue achieves the strongest overall performance among all compared methods except the VLA-based Xiaomi-Robotics-1 [30], which leverages 100,000 hours of real-robot data. The advantage of our model is especially pronounced on unseen tasks, where it substantially surpasses every other WAM-based baseline. This pattern is consistent with the central claim of the report: large-scale embodied video pre-training is most valuable when task and environment distributions shift.

A practical WAM must also be fast enough for closed-loop control. Compared with reactive VLAs, generative WAMs are typically slower because they predict future visual states and often rely on iterative denoising. This latency can create a reactivity gap on contact-rich tasks, where the robot must correct quickly after a failed grasp, or unexpected collision. ZimaBlue addresses this with an asynchronous Slow-Fast control design. The Slow model is a larger WAM trained with more data and capacity, providing strong generalization and world-aware representations. A lightweight Fast module reuses shallow representations from the Slow model to infer actions at a higher control rate. This design preserves the generalization benefits of a large world model while reducing closed-loop latency. Together with diffusion-step distillation and CUDA graph optimization, the system reaches a 33 ms control loop on NVIDIA RTX 4090 in Figure 1.

In summary, this work makes three contributions.

- We present ZimaBlue, framing video scaling as a practical route toward generalizable World Action Models, and provide empirical evidence that scaling video pre-training significantly improves zero-shot generalization and strengthens performance on challenging manipulation benchmarks.
- We introduce a three-stage training framework—comprising egocentric video pre-training, multi-embodiment video-action mid-training, and target-robot post-training—coupled with a unified state-action representation.
- We propose a Slow-Fast WAM architecture that enables 30 Hz closed-loop physical robot control.

2 Related Work

2.1 Vision Language Action Models

Vision-Language-Action (VLA) models have become a central paradigm for generalist robot control. Early large-scale policies such as RT-1 and RT-2 [19, 31] showed that heterogeneous robot demonstrations can be absorbed into a unified language-conditioned policy. More recent systems, including OpenVLA, π_0 , $\pi_{0.5}$, GR00T, and Gemini Robotics [1–5], extend this recipe with pretrained vision-language backbones, diffusion or flow-based action heads, and broader cross-embodiment robot data. By inheriting web-scale visual and semantic priors, these models achieve strong instruction following, object-level generalization, and transfer across diverse manipulation settings.

A complementary line of work uses foundation models as high-level planners rather than end-to-end visuomotor policies. These modular systems decompose robot behavior into semantic reasoning, affordance prediction, task planning, and low-level skill execution [32–35]. Such decomposition improves interpretability and can reuse existing controllers, but it often depends on predefined skill libraries and carefully engineered interfaces between abstract reasoning and physical execution. End-to-end VLAs reduce this hand-engineered modularity by directly mapping observations and language instructions to actions, but they place a heavier burden on action-labeled robot demonstrations to teach spatial, temporal, and physical regularities.

Despite this progress, most VLA models remain primarily action-centric. Their pretrained backbones are typically optimized on static image-text or video-language objectives, while downstream control is learned through supervised imitation over comparatively limited robot trajectories. As a result, VLAs may inherit rich semantic knowledge about *what* to do, but often lack an explicit model of *how* the physical scene evolves under actions. This limitation becomes especially salient in long-horizon manipulation, novel physical skills, contact-rich interaction, and settings where historical context is required to disambiguate the current state. These observations motivate recent efforts to complement VLA-style semantic priors with predictive world-modeling objectives that directly capture temporal dynamics.

2.2 World Action Models

World models learn predictive representations of environment dynamics and have long been studied for model-based control and planning [36–38]. In robotics, they differ mainly in the state representation used for prediction. Latent-space methods learn compact dynamics models for planning or reinforcement learning; 3D methods predict geometric evolution with point clouds or particles; and pixel- or video-space methods directly predict future visual observations [39–43]. Video-based models are particularly attractive for generalist manipulation because they provide dense supervision from every frame transition and can exploit large-scale video pretraining to capture broad spatiotemporal priors.

Recent work has begun to integrate video prediction with robot policy learning. One family follows an *imagine-then-act* paradigm: the model first generates future visual states and then recovers actions through inverse dynamics, planning, or a separate policy conditioned on the predicted future [42–44]. While intuitive, this pipeline can suffer from open-loop drift, mismatch between generated and real observations, and additional latency from test-time video generation. A second family jointly models visual dynamics and actions within a single generative architecture, giving rise to World Action Models (WAMs) [10, 12–14]. By coupling future-frame prediction with action generation, WAMs use visual dynamics as an auxiliary or implicit planning signal for visuomotor control.

Several recent WAMs instantiate this idea at scale. DreamZero [9] frames WAMs as zero-shot policies by adapting pretrained video diffusion models to jointly predict future videos and actions, showing that video-derived physical priors can improve generalization to unseen tasks and new embodiments. LingBot-VA [11] formulates robot control as autoregressive video-action diffusion, interleaving video and action tokens under causal attention and using KV-cache-based history to preserve long-horizon context. LingBot-VA 2.0 [12] further advocates native video-action pretraining for robot control, introducing an embodiment-oriented visual-action tokenizer, causal pretraining, and sparse expert capacity rather than simply adapting a generic video generator. Together, these works suggest that world modeling is not merely an auxiliary representation-learning objective, but can serve as an independent foundation for robot policy learning alongside vision-language pretraining.

However, existing WAMs still face important deployment challenges. First, high-quality video generation is computationally expensive, and naively placing video prediction on the closed-loop control path can reduce control frequency. Second, many joint world-action architectures bind video prediction and action execution to the same short temporal horizon, even though visual world prediction and low-level motor correction naturally operate at different timescales. These limitations motivate architectures that decouple long-horizon predictive reasoning from high-frequency reactive control while preserving information flow between the two.

2.3 Asynchronous Dual-System Policies

Dual-system architectures provide a natural way to reconcile deliberative reasoning with real-time control. In robotics, such systems typically combine a slow but expressive pathway for semantic understanding, planning, or predictive modeling with a fast pathway for reactive action execution. Prior work has explored this principle in several forms: hierarchical manipulation systems and skill-level diffusion planners decompose long-horizon tasks into executable subgoals or skills [45, 46]; dual-policy systems combine a generalist model with a lightweight specialist controller [47–49]; slow-fast sensorimotor policies use fast feedback pathways for contact-rich correction [50]; and asynchronous VLA or action-chunking systems move expensive model inference off the control-critical path [51, 52]. These approaches show that separating slow reasoning from fast execution can improve deployability, especially when the slow model is too expensive to run at the robot control rate.

Recent WAM-based policies extend this principle from semantic planning to predictive world modeling. AHA-WAM [53] explicitly reorganizes WAM inference into a low-frequency video-DiT world planner and a high-frequency action-DiT executor. Its asynchronous horizon-adaptive design reuses long-horizon planner context across multiple action updates, while observation-guided context routing adapts stale planner context to the latest closed-loop observation. This formulation highlights a key insight: the world branch need not generate short-horizon plans at every control step; instead, it can act as a slower predictive substrate that provides reusable context for a faster action policy.

3 Model Architecture

3.1 Unified Representation

Robot datasets expose incompatible control interfaces, ranging from Cartesian end-effector commands for a single arm to bi-manual systems with torso, base, and dexterous-hand controls. We map these native interfaces into a 100-dimensional semantic state-action space so that every coordinate retains consistent physical meaning across embodiments. State and action use the same slot layout: state describes the current robot configuration, whereas action represents a future control chunk.

Semantic slot layout. Each end-effector pose occupies 9 dimensions: 3D translation and a continuous 6D rotation representation. The remaining slots cover grippers, arm joints, torso, mobile base, and dexterous hands, as summarized in Table 1. Each embodiment activates only the slots defined by its native interface. DROID [54], for example, activates the left end-effector, left gripper, and seven left arm joint slots, yielding 17 valid coordinates. Bimanual embodiments activate the corresponding right-arm slots and, when available, torso, base, and hand slots.

Table 1: The semantic layout of the unified 100D state-action interface. Individual embodiments activate only the slots that are physically defined by their native interface, while inactive coordinates are zero-filled and masked.

Slot	Dimension	Description
[0, 9)	9	Left end-effector position and rotation
[9, 10)	1	Left gripper
[10, 19)	9	Right end-effector position and rotation
[19, 20)	1	Right gripper
[20, 27)	7	Left arm joints
[27, 34)	7	Right arm joints
[34, 38)	4	Torso
[38, 54)	16	Mobile base and auxiliary motion channels
[54, 77)	23	Left hand joints
[77, 100)	23	Right hand joints

Chunk-relative action geometry. Absolute coordinates are poorly aligned across robots and scenes. We therefore represent end-effector actions relative to the proprioceptive state at the first frame of each action chunk, which serves as the reference anchor. We denote the anchor pose as $(p_0, R_0) \in \mathbb{R}^3 \times \text{SO}(3)$, and the target end-effector pose as $(p_h, R_h) \in \mathbb{R}^3 \times \text{SO}(3)$ at horizon h . The relative pose is transformed as follows:

$$\Delta p_h = R_0^{-1}(p_h - p_0), \quad \Delta R_h = R_0^{-1}R_h. \quad (1)$$

Here, Δp_h and ΔR_h denote the relative translation and rotation. We parameterize ΔR_h in a 6D rotation format (rot6d) using its first two columns. Following the same principle, joint targets are defined relative to the chunk anchor ($\Delta q_h = q_h - q_0$), where q_0 and q_h correspond to the initial and target joint configurations.

Channel normalization is applied using dataset-wide statistics. Relative translation is normalized to $[-1, 1]$ based on robust percentile bounds, while the 6D rotation representation remains unscaled. At deployment, an inverse transformation restores physical units before relative targets are composed with the current robot state.

Validity mask. Undefined slots are zero-filled and accompanied by state/action validity masks. The masks exclude inactive coordinate supervision, as detailed in the mid-training objective in Section 4.3.2.

3.2 Slow-Fast Dual-System

Figure 2 shows the Slow-Fast architecture used for low-latency WAM control. The design separates high-capacity world modeling from high-frequency action generation. A Slow DiT (5B) maintains a strong video-centric world model and produces intermediate visual dynamics representations, while a lightweight Fast DiT (0.5B) consumes these representations together with the latest robot observation and state to predict the action used for control.

The Slow branch operates on the video-action interface introduced above. Given RGB observations, proprioceptive state s_t , and text instruction ℓ , the system encodes them into visual latent tokens z , state tokens, and language embeddings, respectively. During training, the Slow branch additionally ingests noisy future video tokens alongside noisy action tokens. The primary objective of this branch is to model causal visual dynamics: it predicts future video latents while extracting multi-layer DiT key-value (K/V) features that summarize visual transitions. Notably, auxiliary action supervision is attached to the Slow branch not for runtime deployment, but rather to enforce strong semantic alignment between the predicted video dynamics and the underlying action sequences.

The Fast branch serves as the dedicated action generation module for real-time execution. Rather than executing a high-capacity world model at every control step, the Fast DiT reuses the layer-wise video K/V cache from the Slow branch as conditioning guidance. It leverages updated observation latents, current state tokens, language conditions, and noisy action tokens to predict the final action sequence. This design allows the controller to immediately react to the latest visual and proprioceptive feedback ahead of the Slow branch, while continuously benefiting from its rich spatial-temporal representation.

Mechanistically, the coupling between the two branches occurs directly within the self-attention layer. The Slow DiT computes self-attention over its unified video, action, and state tokens following the causal masking pattern detailed in Section 4.3.2, simultaneously caching the K/V features derived from its video stream at each layer. In the Fast DiT, action queries attend to their own observation, action, and state representations, while concurrently cross-attending to the cached Slow video tokens. As illustrated in Figure 2, this K/V injection effectively bridges the two branches, transferring visual dynamics without the costly explicit generation of full future video sequences. The complete Slow and Fast attention masks are provided in Appendix C.

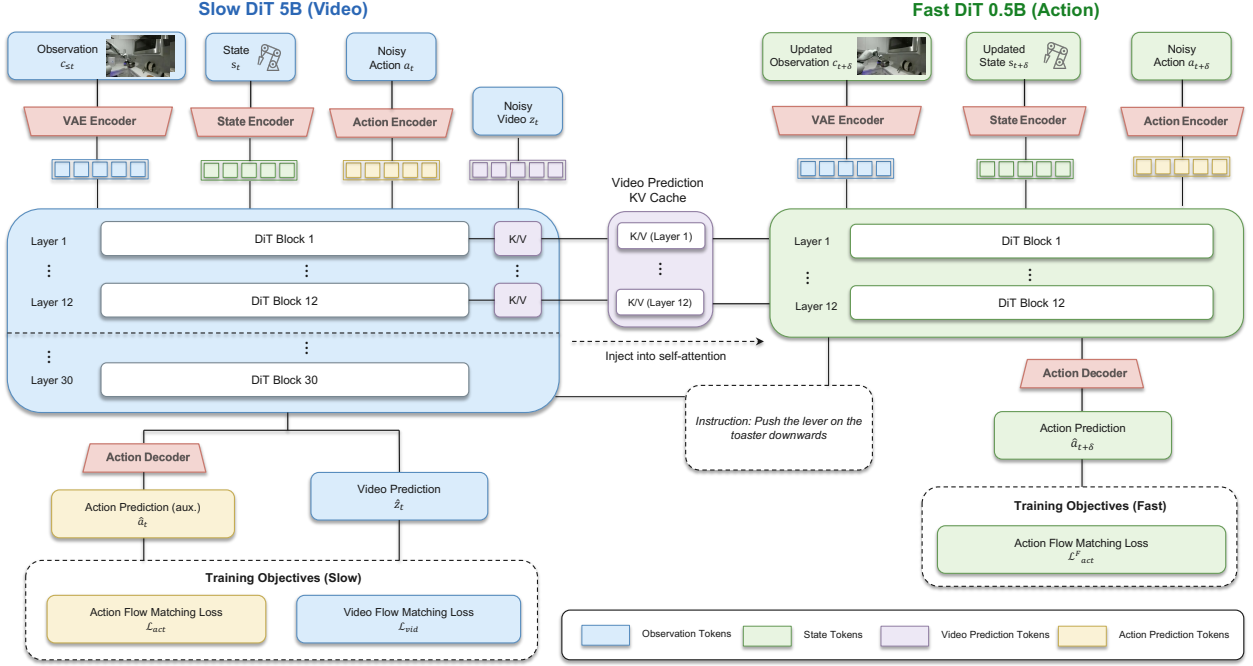


Figure 2: Slow-Fast dual-system architecture. The Slow DiT processes observations, robot state, language instruction, along with noisy video and action tokens, to jointly predict future video latents and their corresponding actions (serves only as auxiliary supervision for video-action alignment). Concurrently, the Fast DiT ingests the updated observation and state, conditioned on the Slow DiT’s video K/V caches, to generate fine-grained final actions for high-frequency closed-loop control.

By default, we instantiate a shallow Fast tower aligned with the early layers of the Slow DiT. Specifically, the Fast branch cross-attends to the video K/V caches from the first 12 Slow layers, injecting local visual and motion priors while retaining the low latency required for high-frequency control. During real-robot deployment (Section 6.1), the Fast branch directly generates the final control actions.

4 Training Pipeline

4.1 Overview

ZimaBlue is trained with a three-level data pyramid, as shown in Figure 3, that progressively adapts a pre-trained text-to-video diffusion transformer to embodied intelligence. The curriculum separates two forms of supervision, *i.e.*, video and action, which are typically entangled in robot learning. The proposed training paradigm has three stages. Specifically, Stage I, *video pretraining*, uses large-scale heterogeneous videos to adapt the visual generative prior to causal embodied dynamics without requiring action annotations. Stage II, *video-action mid-training*, introduces robot trajectories with synchronized observations, proprioceptive states, language instructions, and actions. Rather than treating action prediction as the final objective at this stage, action supervision grounds the Slow world model in control-relevant dynamics and encourages its visual representations to retain motor-relevant information through a unified cross-embodiment action interface. Stage III, *post-training*, specializes the model to a target embodiment and introduces the lightweight Fast branch for responsive closed-loop control.

For training efficiency, we optimize only the Slow branch during video pretraining and video-action mid-training. Since these stages cover the largest and most heterogeneous data mixtures, we concentrate the training budget on learning a strong video-centric world model instead of jointly optimizing both towers throughout the full curriculum. By the start of post-training, the Slow representations have already been grounded by action supervision, substantially simplifying the mapping from visual dynamics to executable actions. We therefore introduce Fast only at this stage, where it can efficiently specialize to the target deployment domain on top of the aligned Slow representation.

Across all stages, we use a unified flow-matching objective. During video pretraining, it is applied only to visual latents, training the Slow branch to model causal forward dynamics. During video-action mid-training, the same objective is extended to action chunks, using action prediction as an auxiliary alignment signal that injects motor relevance into the



Figure 3: Data pyramid. Three-stage training progresses from broad visual diversity to to deployment-specific embodiment. **Stage I (Pre-training)** learns general visual dynamics from diverse video sources without action supervision. **Stage II (Mid-training)** introduces robot actions from multiple embodiments, grounding visual dynamics in cross-embodiment control. **Stage III (Post-training)** specializes the model on target embodiments and benchmarks for deployment.

shared visual representation. During post-training, Fast branch consumes these aligned Slow features together with the latest observation and proprioceptive state to produce low-latency actions for closed-loop control.

4.2 Stage I: Video Pre-training

Video pre-training repurposes generic video generation priors from Wan2.2-TI2V-5B [26] into a causal visual dynamics model for embodied interaction. By learning from large-scale egocentric videos without action annotations, this stage acquires generalizable priors over objects, scenes, motion, and contact that provide a strong representational basis for subsequent action grounding. We elaborate on our recipe from two perspectives: data and training strategy.

4.2.1 Data Recipe

The goal of video pre-training is to obtain an embodied video world model before introducing action supervision. We adopt a two-phase video-only curriculum. First, we train the model on the broadest mixture, covering egocentric human videos, simulated manipulation trajectories, and real robot demonstrations. The source corpus includes EPIC-KITCHENS [55], Egocentric-100K [23], EgoDex [24], HOT3D-Aria [56], DreamDojo [57], GenRobot [58], RoboCOIN [59], DROID [54], AgiBot [60], Galaxea [48], RoboMIND2 [61], InternData-A1 [20]. Egocentric human sources contribute diverse first-person hand-object interaction patterns; simulated sources provide clean and controllable motion sequences; robot sources expose the model to manipulation scenes, camera viewpoints, and object dynamics closer to real-world deployment.

We subsequently continue video pre-training on a curated embodied subset. This dataset excludes noisy web-scale sources that often lack detailed task instructions or suffer from low visual resolution. Instead, we concentrate on manipulation-centric video sources, incorporating datasets such as DreamDojo, RoboCOIN, DROID, AgiBot, Galaxea, RoboMIND2, and InternData-A1, alongside a substantial volume of human egocentric proprietary video. This targeted domain adaptation shifts the video world model toward high-quality robotic viewpoints and refined instruction compliance, all while maintaining the core video-only objective.

The pre-processing retains the task-relevant RGB stream from raw sources, normalizes spatio-temporal scales, and segments long recordings into trainable episodes. Clips undergo random cropping, resizing, and photometric jitter prior to VAE encoding. Each sample comprises a fixed-length visual trajectory, language instructions when available, and null state/action placeholders. Specifically, the visual trajectory is formatted as $8K + 1$ frames across K temporal chunks (with $K = 4$ yielding 33 frames in our default setup). The dummy action stream spans a 24-step horizon but carries no semantic supervision. This unified design establishes a standardized data interface across pre-training and downstream stages within the same causal prediction pipeline.

4.2.2 Training Recipe

To support heterogeneous camera availability, video pre-training uses a unified three-view visual interface. Human egocentric clips typically provide only a primary view, whereas robot demonstrations often contain multiple synchronized views. We therefore represent every sample with a shared three-row vertical canvas: available views are placed in canonical rows, and absent views are padded. This preserves multi-view robot observations while allowing single-view human videos to share the same latent-video representation. A corresponding view-validity mask is propagated through the model. Invalid view regions are excluded from video-token attention and from the flow-matching loss, so padded cameras neither contribute supervision nor introduce spurious context.

For a clean future video latent block z and Gaussian noise ϵ , we form

$$z_t = (1 - \sigma_t^{\text{vid}})z + \sigma_t^{\text{vid}}\epsilon, \quad v^{\text{vid}} = \epsilon - z. \quad (2)$$

The video loss is computed over valid visual tokens,

$$\mathcal{L}_{\text{vid}} = \mathbb{E}_{z, \epsilon, t} \left[\|m^{\text{vid}} \odot (\hat{v}_{\theta}^{\text{vid}}(z_t, c_{\leq t}, \ell) - v^{\text{vid}})\|_2^2 \right], \quad (3)$$

where $c_{\leq t}$ denotes the clean causal video context, ℓ denotes the language condition when available, and m^{vid} combines temporal validity with the view-validity mask introduced by the unified three-view interface. Action and state streams are instantiated only as null placeholders: action supervision $\mathcal{L}_{\text{act}}$ of Slow branch is disabled, and the Fast branch is not involved. To support prediction from variable-length clean contexts, video latents are partitioned into temporal blocks, each with clean and noisy counterparts. A noise level is sampled independently for each block and shared across all tokens within the block to maintain temporal coherence. During training, clean video blocks provide causal context, while their noisy counterparts serve as denoising targets. A block-causal teacher-forcing attention scheme preserves the same temporal dependency structure as autoregressive rollout, while allowing multiple future video blocks to be denoised in parallel. This enables efficient multi-block training without sacrificing the causal information flow required for closed-loop prediction (see Appendix C).

4.3 Stage II: Video-Action Mid-training

Building upon the pre-trained backbone, mid-training incorporates real-robot trajectories using the action encoders and decoders detailed in Section 3. Here, the Slow world model jointly predicts future video latents and action chunks. This joint prediction forces action and video tokens to interact within the shared backbone, grounding pre-trained visual dynamics with cross-embodiment action semantics and producing representations rich in motor-relevant information. Note that during this stage, the Fast branch remains inactive.

4.3.1 Data Recipe

Mid-training introduces a unified cross-embodiment trajectory supervision. Without loss of generality, we select four representative embodiment families—namely DROID [54], AgiBot [60], Galaxea [48], and RoboMIND2-Franka [61]—covering single- and dual-arm manipulation. While each embodiment retains its heterogeneous sensorimotor modalities, all data are projected into a standardized training interface. This design preserves embodiment-specific dynamics while framing the learning process as a unified video-action prediction problem.

Each training sample is anchored at a key timestep and incorporates four synchronized modalities: multi-view video context, proprioceptive state, a future action chunk, and language instructions. To ensure precise cross-modal alignment, the visual context and the 24-step target action chunk cover the exact same temporal horizon starting from the anchor timestep, with video frames sampled at evenly spaced offsets. Language supervision is extracted directly from original annotations, including high-level task instructions and camera view descriptions. Crucially, temporal sampling windows are strictly confined within language-consistent segments to prevent merging disjointed subgoals into a single sample.

Each embodiment is mapped onto the shared three-row vertical canvas according to its sensor configuration: DROID uses two exterior cameras and a wrist camera; AgiBot uses head and left/right hand cameras; Galaxea uses head and left/right wrist cameras; RoboMIND2-Franka uses top and left/right wrist cameras. A shared visual normalization pipeline, including crop, resize, photometric augmentation, and canvas packing, standardizes the input geometry.

As described in Section 3.1, native proprioceptive states and actions are unified into a 100-dimensional space. Specific embodiment families populate corresponding sub-vectors within this representation: DROID occupies a single-arm subset; AgiBot and Galaxea populate bimanual arm, hand/gripper, torso, and mobile-base slots; and RoboMIND2-Franka occupies dual-Franka end-effector, gripper, and joint slots. Each state and action tensor is accompanied by a binary validity mask to handle inactive dimensions.

4.3.2 Training Recipe

Mid-training optimizes a unified video-action flow-matching objective,

$$\mathcal{L}_{sup}^S = \lambda_{\text{vid}} \mathcal{L}_{\text{vid}} + \lambda_{\text{act}} \mathcal{L}_{\text{act}}, \quad (4)$$

where $\mathcal{L}_{\text{vid}}$ is the future video loss and $\mathcal{L}_{\text{act}}$ is the corresponding action loss. For a clean action chunk a and Gaussian noise η , the corrupted action is $a_t = (1 - \sigma_t^{\text{act}})a + \sigma_t^{\text{act}}\eta$, and the target is $v^{\text{act}} = \eta - a$. The video loss is reframed as

$$\mathcal{L}_{\text{vid}} = \mathbb{E}_{z, \epsilon, t} \left[\left\| m^{\text{vid}} \odot (\hat{v}_{\theta}^{\text{vid}}(z_t, a_t, c_{\leq t}, s_t, \ell) - v^{\text{vid}}) \right\|_2^2 \right], \quad (5)$$

and the action loss is calculated as

$$\mathcal{L}_{\text{act}} = \mathbb{E}_{a, \eta, t} \left[\left\| m^{\text{act}} \odot (\hat{v}_{\theta}^{\text{act}}(a_t, z_t, c_{\leq t}, s_t, \ell) - v^{\text{act}}) \right\|_2^2 \right]. \quad (6)$$

Here m^{act} is the action validity mask, and s_t is the proprioceptive state. The mask restricts supervision to physically meaningful dimensions for active coordinates. Invalid slots are zeroed before encoding and are excluded from the loss. We couple the video and action noise levels within a denoising step so that uncertainty in the visual rollout is aligned with uncertainty in the motor command that realizes it.

The block-causal attention pattern coordinates video, action, and state streams, with state tokens serving purely as conditioning inputs. Video tokens comprise clean history context, noisy future tokens, and clean future targets, whereas action tokens consist of noisy future tokens and clean future targets. Crucially, noisy future video and action tokens mutually attend to each other while jointly conditioning on the instruction, state tokens, and clean video history.

This dependency is pivotal for world-action modeling: the video stream generates plausible future observations conditioned on predicted actions, while the action stream infers the control sequences required to drive these visual transitions. During policy inference, the clean target video and action tokens are omitted, enabling the model to progressively denoise their noisy counterparts.

4.4 Stage III: Video-Action Post-training

Post-training specializes the model to a target embodiment by adapting it to the deployment environment, control interface, and task distribution, while preserving the visual dynamics priors acquired during pretraining and the cross-embodiment grounding learned during mid-training. It proceeds in two phases: first, specializing the Slow branch to the target domain, and then training the Fast action branch with Slow frozen.

4.4.1 Target-Domain Slow Branch Specialization

We evaluate our model across two deployment settings: real-robot manipulation on DROID [54] and three simulation benchmarks—LIBERO-Plus [27], RoboTwin-2.0 [28], and RoboCasa365 [29]. All DiT backbone parameters are initialized from the mid-trained checkpoint to retain pre-trained visual dynamics and cross-embodiment grounding. As DROID is present in previous stages, we directly preserve its unified 100-dimensional state and action interface. For the unseen simulation benchmarks, we attach lightweight, embodiment-specific encoders and decoders (see Section 3) to match their native control spaces. Empirically, such an interface-level adaptation converges rapidly without compromising backbone vision-action transfer.

4.4.2 Fast Branch Training

After specializing the Slow branch, we freeze its parameters and train Fast using an action-only flow-matching objective. Fast is initialized from the first 12 layers of the Slow DiT, with all Fast parameters subsequently optimized. Because Fast uses a narrower hidden dimension, we interpolate the transferred weights along the channel dimensions to match its model width. To reproduce the information pattern of asynchronous closed-loop execution, for an action chunk of horizon H , we uniformly sample a time offset $\delta \in \{0, \dots, H-1\}$. Fast receives the updated observation and state (c_{δ}, s_{δ}) and predicts the shifted action chunk $(a_{\delta}, \dots, a_{H-1})$; positions beyond the trajectory are padded and excluded from the loss. Thus, a single Slow prediction can be reused across observations arriving throughout its execution, allowing Fast to continuously close the control loop without waiting for the next Slow rollout.

We denote this shifted action chunk as $\tilde{a} = \text{Pad}(a_{\delta:H})$. To simulate asynchronous replanning while preserving action continuity, we sample a prefix length p and teacher-force the first p actions, treating them as the action prefix already committed by the previous Fast prediction. For action index i , the input to Fast is

$$a_{t,i}^F = \begin{cases} \tilde{a}_i, & i < p, \\ (1 - \sigma_t^{\text{act}})\tilde{a}_i + \sigma_t^{\text{act}}\eta_i, & i \geq p. \end{cases} \quad (7)$$

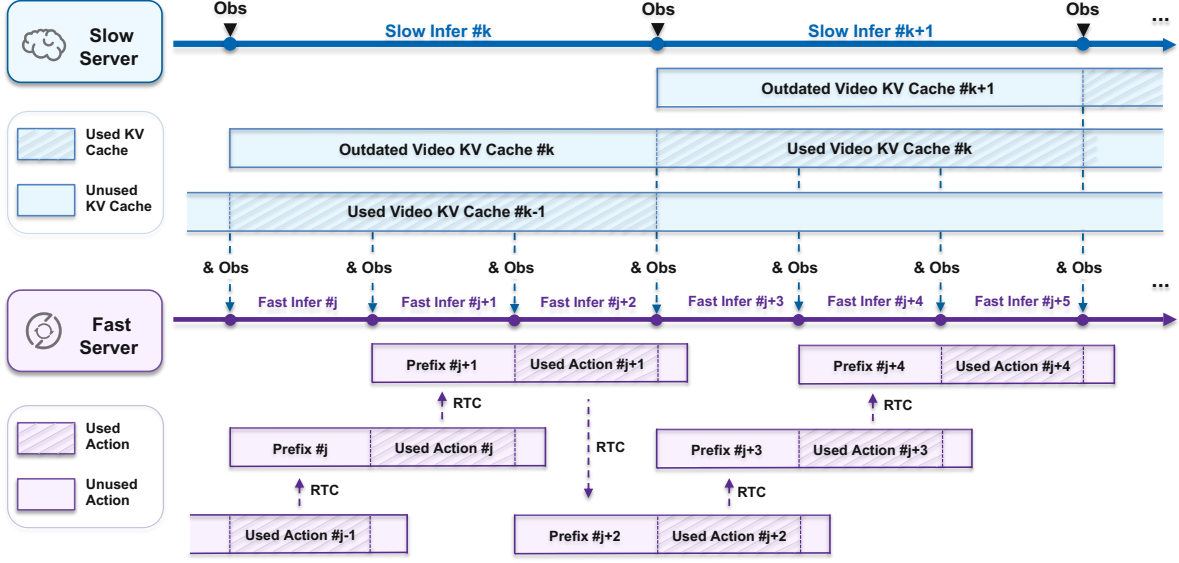


Figure 4: Asynchronous inference in the Slow-Fast dual system. The Slow stream operates at a lower frequency and produces future video K/V caches. The Fast stream runs at a higher frequency, using updated observations, states, and Slow K/V guidance to generate action chunks. The generated actions are continuously executed and provide new observations for subsequent updates. “Outdated” marks a Slow video K/V cache based on past observations that is stale at the current timestep, while “prefix” denotes actions already committed for execution from the previous chunk and retained for continuity. The figure is purely schematic.

The corresponding target velocity is $v^{act,F} = \eta - \tilde{a}$. Thus, the prefix remains clean while only the suffix is noised and predicted. Let $m^{act,F}$ combine the action validity mask m^{act} with additional offset-padding and prefix masks, the action loss is defined as

$$\mathcal{L}_{sup}^F = \mathbb{E}_{a,\eta,t,\delta,p} \left[\left\| m^{act,F} \odot \left(\hat{v}_{\theta}^{act,F} \left(a_t^F, c_{\delta}, s_{\delta}, \mathcal{K}_{slow}, \ell, \delta \right) - v^{act,F} \right) \right\|_2^2 \right]. \quad (8)$$

Here $\mathcal{K}_{slow}$ is the frozen Slow visual guidance and ℓ is the language condition. The corresponding Fast attention pattern is illustrated in Appendix C. The offset exposes Fast to fresh observations after partial execution, while the clean, loss-masked prefix preserves continuity with committed actions. Together they realize training-time Real-Time Chunking (RTC) [52] for fast asynchronous control.

5 Acceleration Schemes

5.1 Overview

ZimaBlue accelerates closed-loop control through three complementary mechanisms: asynchronous Slow-Fast inference, diffusion step distillation, and Torch compile. At deployment, the Slow and Fast branches operate at different frequencies: Slow updates long-horizon world-model guidance asynchronously, while Fast uses the latest observation and available Slow guidance to generate high-frequency actions without waiting for each Slow rollout. Following post-training, Distribution Matching Distillation (DMD) [62] is applied sequentially to the Slow branch’s joint video-action generation and the Fast branch’s action generation, reducing each branch from eight to two DiT evaluations.

5.2 Asynchronous Inference in Dual-System

Figure 4 depicts the asynchronous inference schedule of the Slow-Fast dual system. The design decouples two processes with different temporal requirements: Slow updates long-horizon guidance, while Fast performs high-frequency closed-loop action generation. Instead of waiting for Slow inference to complete, the two streams run concurrently, allowing Fast to continuously refine actions using the latest observations while asynchronously receiving updated Slow guidance.

Slow guidance generation. The Slow stream serves as a low-frequency world-model predictor. Given the observation latent, proprioceptive state, and language condition, it performs a long-horizon rollout and exports the layer-wise video K/V cache. This cache encodes the predicted visual evolution and is reused by multiple Fast requests as temporal guidance. Once a new Slow prediction becomes available, Fast accesses the updated cache without interrupting the ongoing control loop.

Fast closed-loop refinement. The Fast stream performs high-frequency action generation using the latest observation, state, Slow video K/V cache, and noisy action chunk. It predicts an action chunk of horizon H , which is decoded and sent to the robot execution interface. During asynchronous updates, only future unexecuted actions are replaced by newly predicted actions, while the already committed actions remain unchanged.

Temporal consistency via RTC. To improve continuity across consecutive Fast predictions, we adopt an RTC inference mechanism. Each Fast request receives a short prefix from the previous action sequence and predicts the remaining suffix conditioned on the latest observation, state, and Slow guidance. The prefix anchors the transition between consecutive action chunks, while the suffix enables online correction under changing environments.

5.3 Step Distillation

Following target-domain post-training, we apply DMD [62] as a sequential two-stage distillation procedure. We first distill the task-specialized Slow branch from eight to two DiT evaluations per Slow rollout. The resulting distilled Slow branch retains the joint prediction of future video latents and action tokens. We then freeze the distilled Slow branch and distill the Fast action branch from eight to two DiT evaluations per Fast request, conditioned on the layer-wise video K/V guidance from the distilled Slow branch. The resulting distilled Slow–Fast dual system uses two DiT evaluations per invocation of each branch while preserving the original model architecture, action horizon, and asynchronous closed-loop execution protocol.

For each distillation stage, indexed by $b \in \{S, F\}$ for the Slow and Fast branches, respectively, DMD maintains three networks: a deployable student generator G_θ^b , a frozen real-score model D_r^b initialized from the corresponding teacher, and an online fake-score model D_f^b that tracks the evolving student distribution. The student produces a clean sample x_g^b from a truncated two-step rollout. For the Slow branch, $x_g^S = (z_g, a_g)$ contains both future-video latents z_g and an action chunk a_g ; for the action-only Fast branch, $x_g^F = a_g$.

To estimate the distribution mismatch, we sample a noise level $\sigma \sim \mathcal{U}(0.02, 0.98)$ and re-noise x_g^b . The real- and fake-score models then produce the corresponding clean-sample estimates $\hat{x}_{0,r}^b$ and $\hat{x}_{0,f}^b$. Here, $\hat{x}_{0,r}^b$ represents the estimate induced by the frozen teacher distribution, whereas $\hat{x}_{0,f}^b$ represents the estimate induced by the current student distribution. Their normalized difference defines the distribution-matching direction

$$g_{\text{DMD}}^b = \frac{\hat{x}_{0,f}^b - \hat{x}_{0,r}^b}{\text{mean}_{\text{valid}}(|x_g^b - \hat{x}_{0,r}^b|) + \epsilon_{\text{num}}}. \quad (9)$$

The operator $\text{mean}_{\text{valid}}(\cdot)$ averages only over valid, non-masked video or action dimensions, and ϵ_{num} is a small constant for numerical stability. The DMD direction is treated as a stop-gradient target: gradients propagate through the generated sample x_g^b , but not through the two score-model predictions or the normalization term.

We use a truncated two-step UniPC rollout [63] and randomly sample one of the two denoising steps for gradient computation. All preceding steps are evaluated without gradient, so each update backpropagates through only the selected student step. The corresponding shifted noise level is obtained from the scheduler, avoiding reconstruction from discretized timesteps. In both stages, we adopt the two-time-scale optimization scheme of improved DMD [64], updating the fake-score model five times for every generator update to closely track the evolving distribution.

Slow-branch distillation. The Slow student and its real- and fake-score models are initialized from the task-specialized eight-evaluation Slow branch. Because Slow jointly predicts future-video latents and actions, its distillation objective preserves both output spaces. The generator retains the original supervised flow-matching objective $\mathcal{L}_{\text{sup}}^S$ as an optimization anchor and is trained with

$$\mathcal{L}_g^S = \mathcal{L}_{\text{sup}}^S + \mathcal{L}_{\text{DMD},v}^S + \mathcal{L}_{\text{DMD},a}^S, \quad (10)$$

while the online fake-score model is optimized using

$$\mathcal{L}_f^S = \mathcal{L}_{\text{FM},v}^S + \mathcal{L}_{\text{FM},a}^S. \quad (11)$$

The subscripts g and f denote the generator and fake-score objectives, respectively, while the superscripts S and F denote the Slow and Fast branches. The subscript FM denotes flow matching. Here, $\mathcal{L}_{\text{DMD},v}^S$ and $\mathcal{L}_{\text{DMD},a}^S$ apply the direction in Eq. (9) to the generated video and action components, respectively. The terms $\mathcal{L}_{\text{FM},v}^S$ and $\mathcal{L}_{\text{FM},a}^S$ train the fake-score model with standard flow matching on detached Slow-student samples. All loss terms use unit weights. This stage compresses the Slow rollout while preserving the coupling between predicted visual dynamics and their corresponding action trajectories.

Fast-branch distillation. After the distilled Slow branch converges, we freeze it in evaluation mode and detach its layer-wise video K/V features from the Fast optimization graph. For each training sample, the distilled Slow branch performs two DiT evaluations and exports a layer-wise video K/V representation at each denoising step. The Fast branch also performs two DiT evaluations, with its j -th denoising step consuming the Slow guidance exported at the corresponding step j , for $j \in \{0, 1\}$. Unlike the Slow branch, the Fast branch predicts actions only; therefore, x_g^F , $\hat{x}_{0,r}^F$, and $\hat{x}_{0,f}^F$ contain only action tokens. We initialize the Fast student and its real- and fake-score models from the task-specialized eight-evaluation Fast branch and optimize

$$\mathcal{L}_g^F = \mathcal{L}_{\text{sup}}^F + \mathcal{L}_{\text{DMD},a}^F, \quad (12)$$

and

$$\mathcal{L}_f^F = \mathcal{L}_{\text{FM},a}^F. \quad (13)$$

The supervised term $\mathcal{L}_{\text{sup}}^F$ anchors the Fast student to the ground-truth action flow, $\mathcal{L}_{\text{DMD},a}^F$ matches the student action distribution to the frozen Fast teacher, and $\mathcal{L}_{\text{FM},a}^F$ updates the online fake-score model using detached Fast-student actions. Only the Fast student and its online fake-score model are updated during this stage; the distilled Slow conditioner and the Fast real-score model remain frozen. At deployment, all real- and fake-score models are discarded, leaving only the two-step Slow and Fast students for inference.

6 Experiments

6.1 Zero-Shot Real-Robot Evaluation

To validate the individual contributions of egocentric video pre-training and multi-embodiment video-action mid-training, we conduct a series of real-robot evaluations. The evaluation suite consists of 12 zero-shot tasks categorized into two distinct protocols: Standard tasks and Perturbed tasks.

6.1.1 Evaluation Protocol

Evaluation setting. All experiments are conducted on a 7-DoF Franka arm platform. Each ZimaBlue variant is post-trained on the DROID dataset and evaluated on held-out task-scene configurations without task-specific demonstrations. At deployment, the policy receives two external RGB views, one wrist-camera view, the current proprioceptive state, and a language instruction.

Task suites. The benchmark comprises two suites. The *Standard* suite encompasses eight tasks under controlled laboratory conditions, covering language-conditioned target selection, object transport and stacking, articulated-object interaction, and contact-rich appliance manipulation (Figure 5a). The *Perturbed* suite introduces dynamic lighting, background distractors, and novel tabletop appearances across four tasks (Figure 5b), with specific perturbation details provided in Table 2. Detailed task instructions and success criteria are reported in Appendix A.1.

Rollout and metrics. Camera placement, language instructions, scene resets, action decoding, and safety limits are fixed across methods. Each task is evaluated over 10 trials. All tasks use binary success except Toys, which scores three object placements per rollout and therefore contains 30 placement outcomes. Suite results are task-macro averages, so each task contributes equally. Additional rollout details are provided in Appendix A.2.

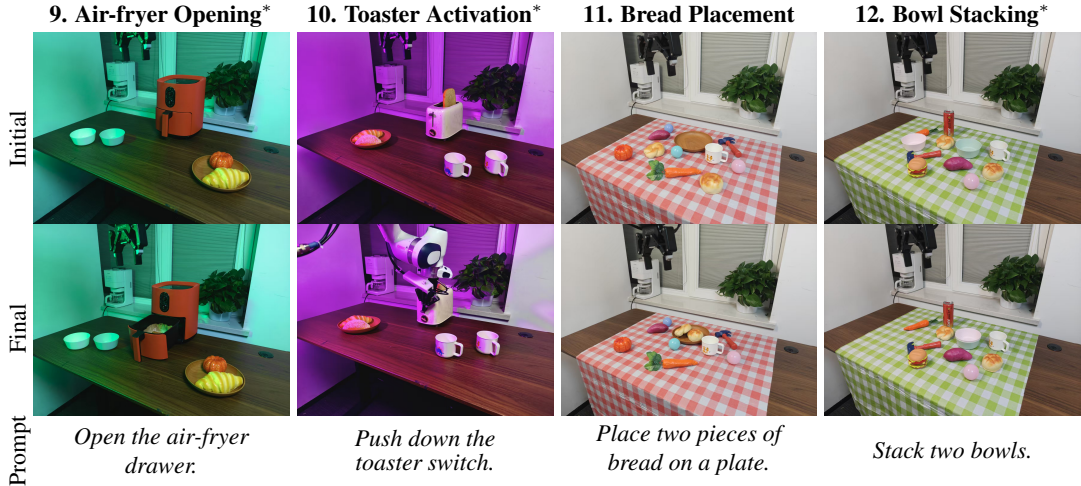
Comparison methods. As illustrated in Table 3, we evaluate four pre-training configurations to investigate the impact of progressive data scaling: (1) baseline, (2) multi-embodiment video-action data, (3) multi-embodiment + 60K hours of egocentric video, and (4) multi-embodiment + 120K hours of egocentric video. Across all variants, the Slow System is post-trained on DROID for 60K steps and subsequently frozen, while the Fast System is trained for 160K steps. All variants use eight denoising steps at inference. To isolate the effects of pre-training data, all models share identical DROID data and optimization schedules during post-training (see Appendix A.3 for implementation details). For external comparison, we evaluate official checkpoints of $\pi_{0.5}$ [5] and DreamZero [9] post-trained on DROID.

6.1.2 Real-Robot Performance Analysis

Performance overview. Table 3 reports suite-level success rates for the four ZimaBlue variants alongside $\pi_{0.5}$ [5] and DreamZero [9]. Under identical DROID post-training, overall success increases monotonically from 36.1% for our Baseline configuration to 77.8% for the full model. Specifically, the full model achieves 87.9% on the Standard suite and 57.5% on the Perturbed suite, outperforming $\pi_{0.5}$ and the 14B-parameter DreamZero by a large margin. Detailed per-task breakdowns are provided in Appendices A.4 and A.5.



(a) Standard real-robot suite.



(b) Perturbed real-robot suite (* denotes perturbed versions of tasks in the standard suite).

Figure 5: Overview of zero-shot real-robot evaluation suites, showing initial/target states and instructions. Top: **Standard suite** (8 tasks) covering spatial grounding, stacking, object transport, articulated objects, and contact-rich control. Bottom: **Perturbed suite** (4 tasks) incorporating environmental shifts—appliance tasks feature dynamic glare, flashing light, and distractors, while bread/bowl tasks introduce unseen tablecloths and novel distractor objects.

Table 2: Perturbed-suite conditions. Tasks are grouped by unseen perturbation type, corresponding to Task 9~12 in Figure 5b.

Task Description	Perturbation Type	Evaluation Condition
<i>Open the air-fryer drawer; Push down the toaster switch</i>	Dynamic Illumination	Glare and flashing lights with surrounding distractor objects
<i>Place two pieces of bread on a plate; Stack two bowls</i>	Scene Appearance	Tablecloth background with unseen distractor objects

Table 3: Zero-shot real-robot success rates (%). Results are task-level macro averages over the Standard, Perturbed, and combined suites. All numeric quantities in the Configuration column are measured in hours. The four ZimaBlue configurations are cumulative and exclude their common DROID post-training. The Baseline is initialized from Wan2.2-TI2V-5B [26]. We use 8-step inference. Both $\pi_{0.5}$ and DreamZero use their official DROID-post-trained checkpoints, matching the target-domain post-training data used by our models. $\pi_{0.5}$ additionally uses web-scale multimodal data. Per-task success counts are provided in Appendices A.4 and A.5.

Policy	Pre-training Data Configuration	Standard (8 tasks)	Perturbed (4 tasks)	Average (12 tasks)
$\pi_{0.5}$ [5]	>10K Robot Data	65.4	32.5	54.4
DreamZero [9]	N/A	61.7	37.5	53.6
ZimaBlue	Baseline	46.7	15.0	36.1
	+ 6K Multi-Embodiment	57.9	22.5	46.1
	+ 6K Multi-Embodiment + 60K Egocentric Videos	82.9	35.0	66.9
	+ 6K Multi-Embodiment + 120K Egocentric Videos	87.9	57.5	77.8

Scaling trends. Action-labeled multi-embodiment trajectories primarily strengthen executable control. Adding 6K hours of such data raises Standard success from 46.7% to 57.9%, Perturbed success from 15.0% to 22.5%, and Overall success from 36.1% to 46.1%. Because each task is evaluated on a limited number of rollouts, per-task results inherently exhibit sampling noise and are not uniformly positive; we therefore place greater weight on consistent suite-level improvements than on individual task fluctuations. The most pronounced gains occur on contact-rich articulated-object manipulation: microwave closing improves from 0/10 to 9/10, and air-fryer opening from 6/10 to 9/10, indicating that cross-embodiment supervision transfers most clearly to demanding execution behaviors.

Egocentric video primarily improves visual generalization and robustness to observation shifts. Adding 60K hours of such data to the multi-embodiment configuration raises Standard success from 57.9% to 82.9%, Perturbed success from 22.5% to 35.0%, and Average success from 46.1% to 66.9%. The corresponding gains include multi-stage tasks that require tracking visual progress, such as bowl stacking (2/10 to 7/10), bread transfer (4/10 to 10/10), and Toys (16/30 to 28/30). Scaling the video corpus from 60K to 120K hours yields a modest 5.0-point gain on Standard, but a striking 22.5-point jump on Perturbed—with all four Perturbed tasks showing marked improvements under unseen backgrounds, distractors, and dynamic illumination. These gains are consistent with broader priors over visual state evolution and task progress driving robustness under visual shifts.

Qualitative analysis. Figure 6 illustrates differences in approach geometry, multi-stage task completion, sustained contact, and robustness in a cluttered Perturbed scene. The stronger configurations more often maintain task progress; however, the remaining failures fall into two groups: failure to advance from a correct intermediate state and local interaction errors such as target displacement, lost contact, or engagement with the tablecloth. Indeed, despite reaching 87.9% on Standard, the full configuration achieves only 5/10–7/10 on each Perturbed task. These cases indicate that broader visual priors should be complemented by progress-aware replanning and responsive corrective control.

6.2 Zero-shot Task Improvement by Dual-System

Protocol. To isolate the performance gains brought specifically by the Slow–Fast Dual-System architecture, we keep all configurations unchanged and vary only the deployment model architecture, yielding two variants: Slow and Slow–Fast. Both variants are evaluated on the eight Standard and four Perturbed real-robot tasks defined in Section 6.1. All policies are post-trained on the same DROID split for 60k optimization steps. The Slow policy operates at a lower closed-loop frequency, generating a 24-step action chunk. In contrast, the dual-system Fast branch runs in real time, conditioning on the latest observation while leveraging the predictive context from the Slow branch. This enables dynamic error correction and real-time contact adjustments without invoking a full world-model rollout.

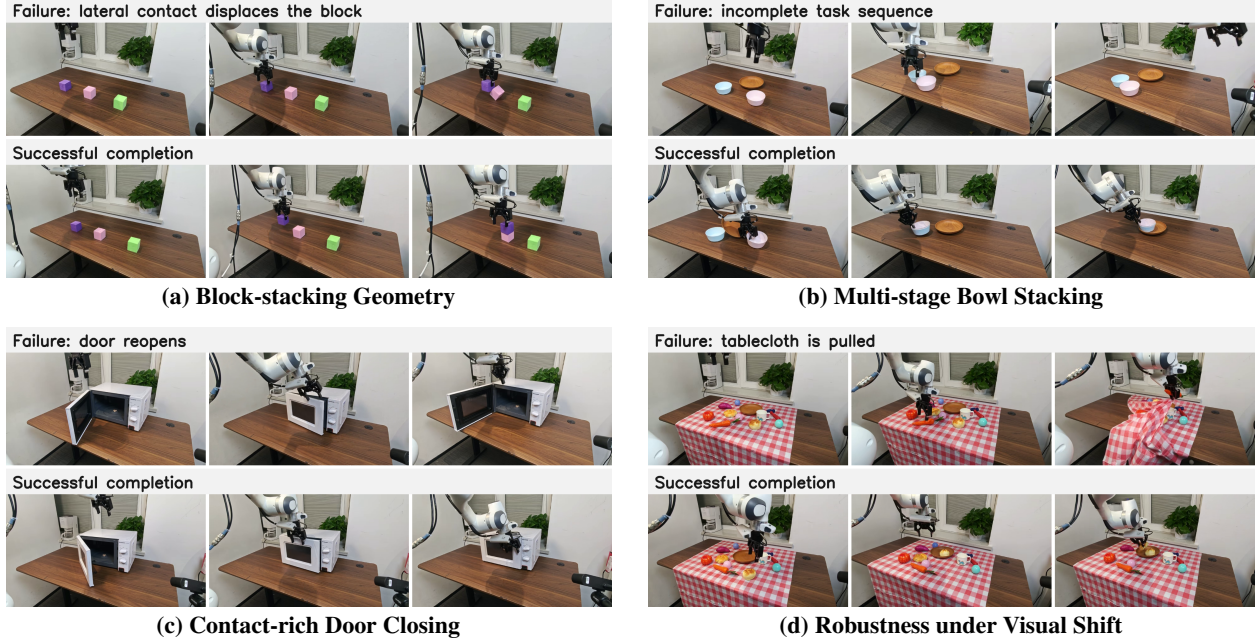


Figure 6: Representative real-robot rollouts, ordered from left to right. Panel (a) contrasts a lateral placement failure with successful top-down placement under the same pink-target instruction. Panel (b) contrasts an incomplete bowl sequence with placement of the completed stack on the plate. Panel (c) contrasts a door that reopens after contact with a complete microwave closure. Panel (d) contrasts pulling the tablecloth with completed bread placement in the cluttered Perturbed scene.

Results and analysis. We report per-task success rates in Table 4, which provide a more diagnostic view of Dual-System behavior. The breakdown separates short-horizon spatial grounding from contact-rich, articulated-object, and multi-object manipulation tasks, where failures often stem from delayed visual correction, inaccurate contact timing, or stale scene-state estimates. This per-task analysis reveals whether Dual-System improves robustness across the held-out suite or mainly raises the average by solving a small subset of easier tasks.

The task-level results indicate that the Dual-System is most effective on tasks that demand frequent visual feedback and recovery from changing scene states. On the Standard suite, it improves Basket, Air Fryer, Toys, and Microwave, spanning spatial grounding, articulated-object interaction, and long-horizon multi-object execution. These gains increase the task-macro average from 80.8% to 87.9%. The improvement is more pronounced under the more challenging Perturbed setting, where the task-macro average increases from 30.0% to 57.5%, compared with a 7.1 percentage-point gain on the Standard suite. Such settings are particularly sensitive to stale observations and delayed corrections, making them well suited to the Fast branch’s online RGB-conditioned updates. Overall, these results highlight the value of the Dual-System for improving closed-loop execution on challenging tasks that require timely visual correction.

Acceleration Ablation. We evaluate deployment efficiency by comparing the Slow policy, the asynchronous Dual-System, and the DMD-distilled Dual-System with compilation. As reported in Table 4, asynchronous execution reduces end-to-end inference latency from 449.6 ms to 145.6 ms, corresponding to a $3.1\times$ speedup over the Slow policy. Distillation and compilation further reduce latency to 33.0 ms, yielding a $4.4\times$ speedup over the undistilled Dual-System and a $13.6\times$ speedup overall. The accelerated model retains a 75.0% overall task-macro success rate, 2.8 percentage points below the undistilled Dual-System.

6.3 Benchmark Results

We evaluate ZimaBlue across three representative simulation benchmarks—LIBERO-Plus [27], RoboTwin-2.0 [28], and RoboCasa365 [29]—to comprehensively assess key dimensions of generalist manipulation. Specifically, LIBERO-Plus assesses robustness against observation, instruction, and environmental perturbations; RoboTwin 2.0 focuses on bimanual manipulation under domain randomization; and RoboCasa365 evaluates large-scale performance on both seen and unseen household tasks. For simplicity, we conduct all evaluations exclusively on the Slow System.

Table 4: Ablation of inference latency and task performance. Per-task zero-shot success counts and task-macro average success rates are reported under the Standard and Perturbed real-robot evaluation settings. End-to-end inference latency is reported for the 8-step Slow and Dual-System, as well as 2-step DMD-distilled variants. The DMD-distilled Dual-System includes compilation.

Setting	Task	Category	Slow	Dual-System	DMD-Distilled
Standard	Bowls	Stacking	10/10	8/10	8/10
	Basket	Spatial Grounding	6/10	10/10	9/10
	Bread	Object Transfer	10/10	9/10	9/10
	Blocks	Spatial Stacking	9/10	7/10	8/10
	Air Fryer	Articulated Object	8/10	10/10	9/10
	Toys	Multi-Object Transport	23/30	28/30	27/30
	Microwave	Articulated Object	6/10	9/10	9/10
	Toaster	Contact-Rich Control	8/10	8/10	7/10
Average (%) $\uparrow$ 8 tasks			80.8	87.9	85.0
Perturbed	Bowls	Stacking	6/10	5/10	6/10
	Bread	Object Transfer	2/10	7/10	5/10
	Air Fryer	Articulated Object	2/10	5/10	7/10
	Toaster	Contact-Rich Control	2/10	6/10	4/10
Average (%) $\uparrow$ 4 tasks			30.0	57.5	55.0
Overall Average (%) $\uparrow$ 12 tasks			63.9	77.8	75.0
Latency (ms) $\downarrow$			449.6	145.6	33.0

Table 5: Evaluation results on LIBERO-Plus. “Average” represents the unweighted arithmetic mean of the success rates across all seven perturbation types. Within each evaluation protocol, the best and second-best performances are highlighted in **bold** and underlined, respectively.

Model	Camera	Robot	Language	Light	Background	Noise	Layout	Average
<i>Zero-shot Transfer</i>								
Fast-WAM [66]	16.4	44.5	68.9	78.2	53.7	37.7	60.7	51.5
LingBot-VA [11]	40.9	83.0	86.4	82.3	53.1	64.4	76.2	69.5
$\pi_{0.5}$ [5]	78.4	73.6	80.8	96.2	94.1	89.0	84.5	85.2
ImageWAM-9B [67]	79.8	58.7	95.2	96.1	91.2	<u>93.3</u>	83.1	85.3
ABot-M0.5 [68]	70.5	87.4	88.6	94.0	89.7	75.5	<u>85.2</u>	84.4
InternVLA-A1.5 [69]	83.1	55.1	86.9	<u>96.4</u>	98.2	95.6	<u>85.2</u>	<u>85.8</u>
ZimaBlue (Ours)	58.1	88.9	<u>91.5</u>	98.0	91.5	93.1	86.1	86.7
<i>Supervised Fine-tuning</i>								
π_0 [4]	79.6	21.1	72.5	84.7	86.2	68.3	69.4	68.8
GR00T-N1.6 [3]	92.6	33.5	80.1	93.6	95.4	93.6	75.0	80.5
OpenVLA-OFT+PT [70]	92.8	30.3	<u>85.8</u>	94.9	93.9	89.3	77.6	80.7
ACoT-VLA [71]	96.6	70.4	79.7	95.1	<u>97.1</u>	<u>95.9</u>	<u>85.0</u>	88.5
CAC-VLA [72]	91.2	78.4	83.3	<u>97.5</u>	<u>97.1</u>	95.4	87.8	<u>90.1</u>
ZimaBlue (Ours)	<u>95.4</u>	81.1	88.8	98.8	99.2	96.5	84.3	92.0

6.3.1 LIBERO-Plus

LIBERO-Plus [27] assesses policy robustness across seven controlled perturbation dimensions: camera viewpoints, robot initial states, language instructions, lighting conditions, background textures, sensor noise, and object layouts (visualized in Appendix B.1). We evaluate models under two settings: zero-shot transfer—where policies are fine-tuned strictly on standard LIBERO [65]—and supervised fine-tuning (SFT) on the LIBERO-Plus training set. For a fair comparison, baselines are restricted to representative methods with publicly available code/checkpoints or official leaderboard records.

As illustrated in Table 5, ZimaBlue achieves an overall success rate of 86.7% in the zero-shot setting, outperforming InternVLA-A1.5 [69] by 0.9 percentage points. It demonstrates particularly high robustness to initial robot state and lighting variations, though camera viewpoint changes represent a key bottleneck. With task-specific SFT, ZimaBlue improves to 92.0%, surpassing the second-best method, CAC-VLA [72], by 1.9 points under identical training protocols.

Table 6: Evaluation results on the RoboTwin 2.0 benchmark. Success rates (%) are averaged over 50 tasks for both “Clean” and “Randomized” settings, with “Average” denoting their mean. The best and second-best results are highlighted in **bold** and underlined.

Model	Clean	Randomized	Average
<i>Vision-Language-Action Models</i>			
$\pi_{0.5}$ [5]	82.7	76.8	79.8
Qwen-VLA [73]	86.1	87.2	86.7
ABot-M0 [74]	86.1	85.1	85.6
InternVLA-A1 [7]	89.4	89.6	89.5
StarVLA- α [75]	88.7	87.8	88.3
Qwen-RobotManip [76]	93.7	94.0	93.9
InternVLA-A1.5 [69]	<u>93.3</u>	<u>93.0</u>	<u>93.2</u>
<i>World Action Models</i>			
Motus [77]	88.7	87.0	87.8
LingBot-VA [11]	92.9	91.6	92.2
Fast-WAM [66]	91.9	91.8	91.8
FlowWAM [78]	92.9	92.1	92.5
LingBot-VA 2.0 [12]	93.8	93.4	93.6
ABot-M0.5 [68]	<u>94.0</u>	<u>94.2</u>	<u>94.1</u>
ZimaBlue (Ours)	94.7	94.3	94.5

Specifically, it achieves top-ranking performance across five perturbation types (initial states, language instructions, lighting, background textures, and sensor noise) while maintaining competitive accuracy on camera viewpoints and object layouts. Notably, the 5.3-point overall gain from SFT is primarily driven by a dramatic surge in camera viewpoint robustness (from 58.1% to 95.4%). This gap indicates that zero-shot viewpoint generalization is constrained by pre-training data diversity, marking a key target for future scaling.

6.3.2 RoboTwin 2.0

RoboTwin 2.0 [28] comprises 50 bimanual manipulation tasks evaluated under two conditions: Clean (standard environments) and Randomized (featuring substantial variations in background, lighting, object placement, and table appearance). Following the multi-task training protocol of recent video-action models [11, 66, 68], we post-train a single policy model across all 50 tasks using a mixture of clean and randomized demonstrations. A single policy checkpoint is evaluated across all tasks, with success rates computed over 100 trials per task under each condition.

As presented in Table 6, ZimaBlue achieves success rates of 94.7% in the Clean setting and 94.3% under Randomized evaluation, yielding an Average success rate of 94.5%. ZimaBlue consistently outperforms all compared VLA and WAM methods across Clean, Randomized, and Average metrics. Notably, compared to the strongest prior WAM baseline, ABot-M0.5 [68], ZimaBlue delivers improvements of 0.7 percentage points (pp) on Clean, 0.1 pp on Randomized, and 0.4 pp on Average. Furthermore, the minimal gap of only 0.4 pp between Clean and Randomized performance demonstrates that ZimaBlue effectively preserves its manipulation capability under substantial visual domain randomization. Detailed per-task success rates are reported in Appendix B.2.

6.3.3 RoboCasa365

RoboCasa365 [29] evaluates generalist robot policies across 365 everyday manipulation tasks and 2,500 diverse kitchen environments. We train our policy on the pre-training Human300 split, and evaluate it under the standard 50-task multi-task protocol. The 50 target tasks comprise 18 Atomic-Seen, 16 Composite-Seen, and 16 Composite-Unseen tasks. Atomic-Seen measures the execution of individual skills; Composite-Seen evaluates long-horizon compositions observed during training, and Composite-Unseen tests zero-shot generalization to held-out composite task templates. Each task is evaluated over 50 rollouts. We report the average success rate for each split along with the trial-weighted Overall score across all 50 tasks.

As shown in Table 7, ZimaBlue achieves the highest Average, Composite-Seen, and Composite-Unseen success rates among all WAM methods, ranking second only to the VLA-based Xiaomi-Robotics-1 [30]. Notably, while Xiaomi-Robotics-1 relies on 100,000 hours of real-world robot data, ZimaBlue attains highly competitive performance using only 6,000 hours action-labeled robot trajectories. Compared to ABot-M0.6 [79], ZimaBlue boosts Average success by 2.9 pp and Composite-Seen by 2.1 pp. Crucially, on Composite-Unseen tasks, ZimaBlue more than doubles the success rate of ABot-M0.6 from 7.9% to 16.5%. This substantial advantage on unseen long-horizon tasks validates

Table 7: Evaluation on RoboCasa365 [29] benchmark. “Average” denotes success rates (%) across three task. The best and second-best results are highlighted in **bold** and underlined. * denotes results reported in the original paper; all other baseline results are taken from the official RoboCasa365 leaderboard [79].

Model	Atomic-Seen	Composite-Seen	Composite-Unseen	Average
<i>Vision-Language-Action Models</i>				
Diffusion Policy [80]	15.7	0.2	1.3	6.1
π_0 [4]	34.6	6.1	1.1	14.8
$\pi_{0.5}$ [5]	39.6	7.1	1.2	16.9
GR00T-N1.5 [3]	50.7	14.8	2.7	23.9
GR00T-N1.6 [3]	51.1	9.4	1.7	21.9
Qwen-RobotManip* [76]	68.6	20.1	<u>14.9</u>	35.9
RLDX-1 [81]	67.6	<u>27.9</u>	<u>8.5</u>	<u>36.0</u>
Xiaomi-Robotics-1 [30]	80.2	57.1	32.1	57.4
<i>World Action Models</i>				
GigaWorld-Policy 0.1 [14]	44.4	11.8	2.9	20.7
WorldDreamer [82]	66.3	26.7	9.0	35.3
ABot-M0.5 [68]	75.6	37.7	3.3	40.3
ABot-M0.6 [79]	79.4	<u>48.3</u>	<u>7.9</u>	<u>46.6</u>
ZimaBlue (Ours)	<u>78.1</u>	50.4	16.5	49.5

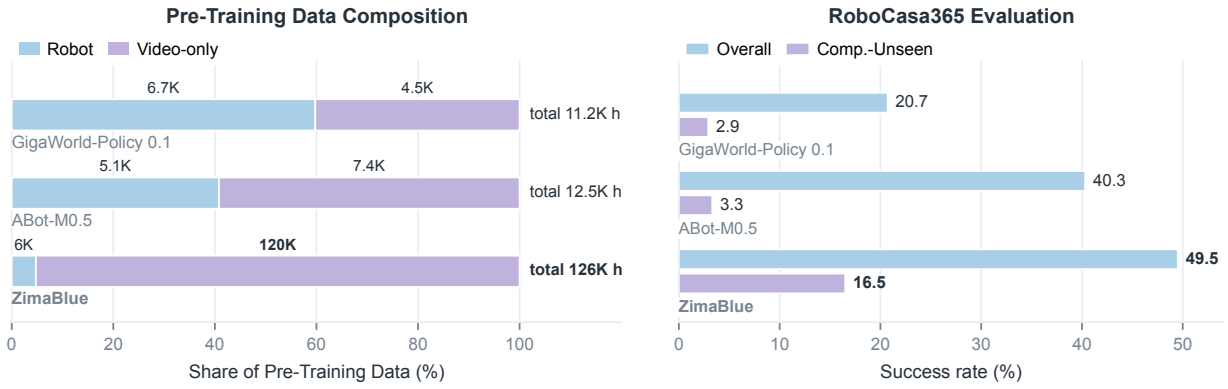


Figure 7: Pre-training Data Composition and Robocasa365 Performance. **Left:** Comparison of pre-training data volume and ratio between robot trajectories and video-only data across different WAMs. **Right:** The results on RoboCasa365 demonstrate a direct positive correlation between pre-training video scale and success rates, particularly on Composite-Unseen tasks.

the effectiveness and generalization capabilities of our pre-training scheme. Detailed per-task success rates for all 50 evaluation tasks are reported in Appendix B.3.

To evaluate the efficacy of video pre-training, we compare the pre-training data compositions and evaluation performance across different WAMs in Figure 7. All three methods utilize a comparable scale of real-robot demonstration data, ranging from 5.1K to 6.7K hours. However, ZimaBlue incorporates 120K hours of video-only data—over 16 times that of ABot-M0.5 [68] and 26 times that of GigaWorld-Policy 0.1 [14]. This massive scale-up in action-free video pre-training translates directly to substantial performance gains, boosting the Composite-Unseen success rate to 16.5% (versus 3.3% and 2.9%). These results highlight the pivotal role of large-scale video pre-training in enabling strong generalizability to unseen scenes.

To qualitatively illustrate the impact of video pre-training, Figure 8 compares execution trajectories on a complex, long-horizon task. The task requires grounding two relational object descriptions alongside two ordinal spatial targets. ZimaBlue accurately identifies the container holding the chicken drumstick, places it onto the second-highest freezer rack, and subsequently places the vegetable container onto the top rack. In contrast, the ablated policy without video pre-training mistakenly targets the chicken drumstick itself rather than its container. Even after failing the grasp, it prematurely advances to the placement stage, demonstrating a compound failure of referential grounding and subtask execution verification.

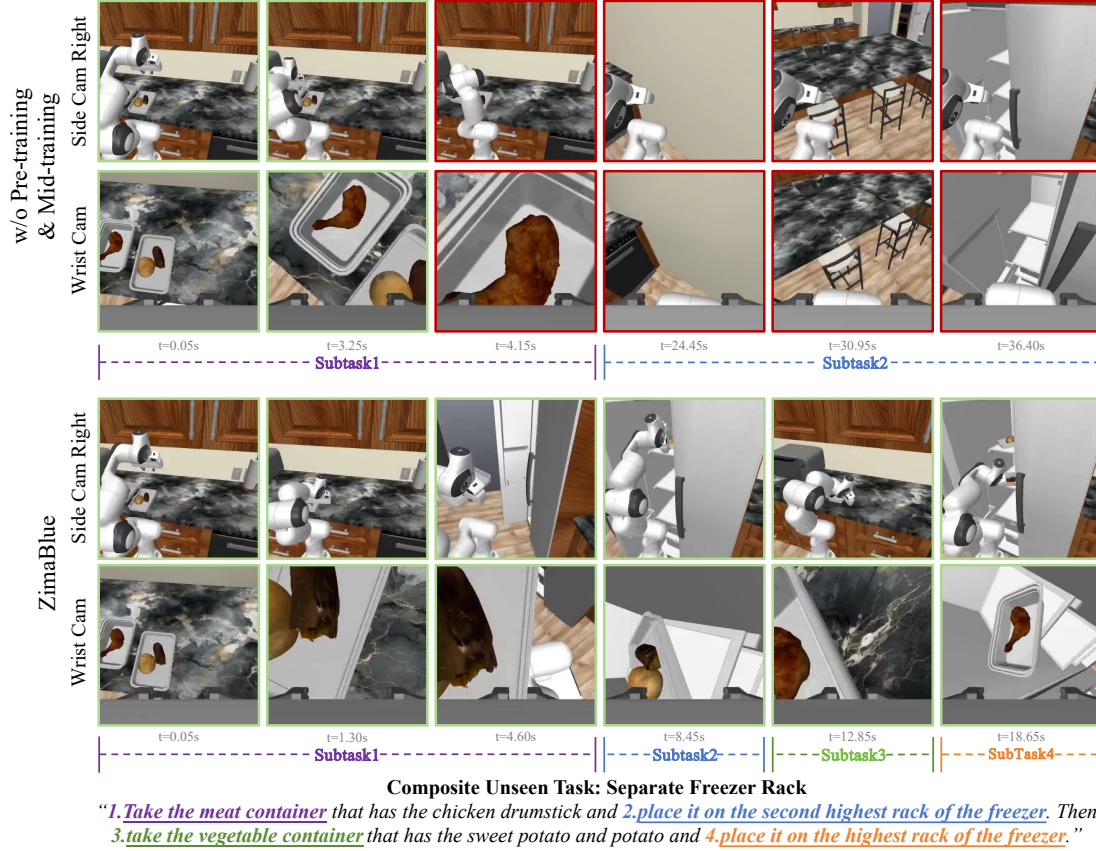


Figure 8: Qualitative comparison on a Robocasa365 [29] Composite-Unseen task. Green boxes highlight instruction-consistent actions, whereas red boxes denote execution errors. ZimaBlue successfully places both containers onto the designated freezer racks. In contrast, the ablated policy targets an incorrect object and prematurely proceeds to the next stage despite a failed grasp.

7 Conclusion and Future Work

We present ZimaBlue, a scalable World Action Model that positions embodied video as a primary axis for scaling generalizable robot learning. Through a structured pipeline—large-scale egocentric pre-training, cross-embodiment video-action alignment, and target-robot post-training—ZimaBlue acquires robust physical and dynamic priors that boost task and environmental generalization. To make these large generative priors practical, our Slow-Fast dual system decouples high-level world reasoning from real-time closed-loop control. Zero-shot real-robot evaluations and simulation benchmark results demonstrate that ZimaBlue offers a viable paradigm for scaling embodied intelligence: *learning broadly from videos, and acting efficiently via decoupled reasoning and control.*

The 120,000 hours of video in this work offer merely a glimpse into the full potential of scaling for robotics. We aim to push this frontier forward along four key directions. First and most importantly, expanding evaluation is key. Developing diverse, multi-embodiment benchmarks across rich task suites will allow us to thoroughly stress-test and quantify the model’s true capabilities. Second, we will scale up pre-training across both data and model dimensions. Beyond incorporating vastly larger and more diverse egocentric human video, expanding model capacity will be essential to building a truly generalist, physics-aware foundation model. Third, endowing the system with stronger reasoning capabilities—such as through higher-level reasoning modules—can unlock complex, long-horizon tasks by enabling hierarchical planning and dynamic self-correction. Finally, bridging the gap toward rapid skill acquisition in novel environments calls for in-context learning. By treating physical demonstrations directly as prompt inputs, the robot can internalize and execute new behaviors at inference time—entirely without gradient updates.

8 Authors

Core Contributors

Xionghao Wu^{1,3,4,7}, Yijun Yang^{1,2,3,4}, Shiyang Zhou^{1,3,4}, Haoze Sun^{1,4,5,7}, Jianhui Liu^{4,5,7}, Songsong Yu^{4,6,7}, Jiyao Zhang⁸, Wenbo Li⁹ 

Listed by task with no indication of priority: ¹Data, ²Pre-training, ³Mid-training, ⁴Post-training, ⁵Dual-System, ⁶Acceleration, ⁷Deployment & Demo, ⁸Guidance, ⁹Project Lead.

Contributors

Bo Wang, Guoqing Ma, Lin Song, Renjie Liao, Shenghe Zheng, Wei Tang, Xiaojuan Qi, Yanwei Li, Yuan Zhang, Zhuotao Tian

Listed in alphabetical order.

Supervision

Haoyang Huang, Nan Duan

 Corresponding author: fenglinglwb@gmail.com

References

- [1] Gemini Robotics Team. Gemini robotics: Bringing AI into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [2] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P. Foster, Pannag R. Sanketi, Quan Vuong, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2025.
- [3] Johan Bjorck, Noah Brown, Chuning Zhu, Jiannan Xiang, Aman Ahuja, Fengyuan Hu, Grace Wang, Oleg Sushkov, Danfei Xu, Shuran Song, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [6] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, et al. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026.
- [7] Junhao Cai, Zetao Cai, Jiafei Cao, Yilun Chen, Zeyu He, Lei Jiang, Hang Li, Hengjie Li, Yang Li, Yufei Liu, et al. Internvla-a1: Unifying understanding, generation and action for robotic manipulation. *arXiv preprint arXiv:2601.02456*, 2026.
- [8] Senqiao Yang, Chengyao Wang, Yuxin Chen, Zixuan Wang, Longxiang Tang, Haokun Gui, Jinhui Ye, Changsheng Lu, Xiaoyang Wu, Mingkang Zhu, Pengguang Chen, Shu Liu, Zhuotao Tian, Hengshuang Zhao, Bei Yu, and Jiaya Jia. Beyond data scaling: Representation-centric continued pre-training for vision-language-action models, 2026. URL <https://arxiv.org/abs/2608.27550>.
- [9] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [10] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.
- [11] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, Yujun Shen, and Yinghao Xu. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [12] Qihang Zhang, Lin Li, Luyao Zhang, Shuai Yang, Yiming Luo, Shuaiting Li, Ruilin Wang, Junke Wang, Jiahao Shao, Gangwei Xu, et al. Native video-action pretraining for generalizable robot control. *arXiv preprint arXiv:2607.08639*, 2026.
- [13] MotuBrain Team, Chendong Xiang, Fan Bao, Haitian Liu, Hengkai Tan, Hongzhe Bi, James Li, Jiabao Liu, Jingrui Pang, Kiro Jing, et al. Motubrain: An advanced world action model for robot control. *arXiv preprint arXiv:2604.27792*, 2026.
- [14] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, et al. Gigaworld-policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026.
- [15] Dyna Robotics. Dyna-2: A 1-million-hour scaling law for world-action models. August 2026. URL <https://dyna.co/dyna-2>.
- [16] Cheng Chi, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. *arXiv preprint arXiv:2402.10329*, 2024.

- [17] Omar Rayyan, John Abanes, Mahmoud Hafez, Anthony Tzes, and Fares Abu-Dakka. Mv-umi: A scalable multi-view interface for cross-embodiment learning. *IEEE Access*, 2026.
- [18] Zhaxizhuom Zhaxizhuoma, Kehui Liu, Chuyue Guan, Zhongjie Jia, Ziniu Wu, Xin Liu, Tianyu Wang, Shuai Liang, Pengan Chen, Pingrui Zhang, et al. Fastumi: A scalable and hardware-independent universal manipulation interface with dataset. In *Conference on Robot Learning*, pages 3069–3093. PMLR, 2025.
- [19] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems*, 2023.
- [20] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, Xu Shi, Ning Gao, Hangxu Liu, Xuekun Jiang, Zherui Qiu, Feng Yuan, et al. Interndata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 976–985, 2026.
- [21] Kristen Grauman, Andrew Westbury, Lorenzo Torresani, Kris Kitani, Jitendra Malik, Triantafyllos Afouras, Kumar Ashutosh, Vijay Baiyya, Siddhant Bansal, Bikram Boote, et al. Ego-exo4d: Understanding skilled human activity from first-and third-person perspectives. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19383–19400, 2024.
- [22] Ropedia. Xperience-10m: A large-scale egocentric multimodal dataset with structured 3d/4d annotations, 2026. Dataset.
- [23] Build AI. Egocentric-100k, 2025. URL <https://huggingface.co/datasets/buildai/egocentric-100k>.
- [24] Ryan Hoque, Peide Huang, David Yoon, Jian Zhang, et al. Egodex: Learning dexterous manipulation from large-scale egocentric video. In *International Conference on Learning Representations*, volume 2026, pages 4218–4237, 2026.
- [25] Ruijie Zheng, Dantong Niu, Yuqi Xie, Jing Wang, Mengda Xu, Yunfan Jiang, Fernando Castañeda, Fengyuan Hu, You Liang Tan, Letian Fu, et al. Egoscale: Scaling dexterous manipulation with diverse egocentric human data. *arXiv preprint arXiv:2602.16710*, 2026.
- [26] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Fei Wu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [27] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [28] Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [29] Soroush Nasiriany, Sep Nasiriany, Abhiram Maddukuri, and Yuke Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations*, volume 2026, pages 98643–98667, 2026.
- [30] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.
- [31] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023.
- [32] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning*, 2022.
- [33] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. PaLM-E: An embodied multimodal language model. In *International Conference on Machine Learning*, 2023.

- [34] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.
- [35] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation*, 2023.
- [36] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019.
- [37] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *International Conference on Learning Representations*, 2020.
- [38] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. DreamerV3: Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [39] Mahmoud Assran et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [40] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3d world models for in-the-wild robotic manipulation. *arXiv preprint arXiv:2507.06450*, 2025.
- [41] Haochen Shi, Huazhe Xu, Samuel Clarke, Yunzhu Li, and Jiajun Wu. RoboCraft: Learning to see, simulate, and shape elasto-plastic objects with graph networks. *Robotics: Science and Systems*, 2022.
- [42] Yilun Du, Mengjiao Yang, Pete Florence, Fei Xia, Ayzaan Wahid, Brian Ichter, Pierre Sermanet, Tianhe Yu, Pieter Abbeel, Joshua B. Tenenbaum, et al. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems*, 2023.
- [43] Gaoyue Zhou, Yecheng Jason Hong, Qi Wu, Panpan Wu, Haoqi Zhou, Jie Wang, Li Yi, and Chuang Gan Wu. RoboDreamer: Learning compositional world models for robot imagination. *arXiv preprint arXiv:2404.12377*, 2024.
- [44] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025.
- [45] Tianshuo Yang, Guanyu Chen, Yutian Chen, Zhixuan Liang, Yitian Liu, Zanzin Chen, Chunpu Xu, Haotian Liang, Jiangmiao Pang, Yao Mu, et al. HiVLA: A visual-grounded-centric hierarchical embodied manipulation system. *arXiv preprint arXiv:2604.14125*, 2026.
- [46] Zhixuan Liang, Yao Mu, Hengbo Ma, Masayoshi Tomizuka, Mingyu Ding, and Ping Luo. Skilldiffuser: Interpretable hierarchical planning via skill abstractions in diffusion-based task execution. In *CVPR*, 2024.
- [47] Qingwen Bu et al. RoboDual: A dual-system vla model for robotic manipulation. In *Conference on Robot Learning*, 2024.
- [48] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianing Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.
- [49] Hao Chen, Jiaming Liu, Chenyang Gu, Zhuoyang Liu, Renrui Zhang, Xiaoqi Li, Xiao He, Yandong Guo, Chi-Wing Fu, Shanghang Zhang, et al. Fast-in-slow: A dual-system foundation model unifying fast manipulation within slow reasoning. *arXiv preprint arXiv:2506.01953*, 2025.
- [50] Zixuan Xue, Cheng Chi, Ziyu Jia, Xuehai Xu, Zhongzheng Ren, Chuer Lin, Hao-Shu Fang, and Cewu Ji. Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation. In *Robotics: Science and Systems*, 2025.
- [51] Noriaki Hirose et al. AsyncVLA: Asynchronous vision-language-action model via delayed fusion. *arXiv preprint arXiv:2602.07463*, 2026.
- [52] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *NeurIPS*, 2026.

- [53] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, Ru Ying, Ran Zheng, and Yao Mu. AHA-WAM: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026.
- [54] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [55] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Sanja Fidler, Antonino Furnari, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, et al. The epic-kitchens dataset: Collection, challenges and baselines. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):4125–4141, 2020.
- [56] Prithviraj Banerjee, Sindi Shkodrani, Pierre Moulon, Shreyas Hampali, Shangchen Han, Fan Zhang, Linguang Zhang, Jade Fountain, Edward Miller, Selen Basol, et al. Hot3d: Hand and object tracking in 3d from egocentric multi-view videos. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7061–7071. IEEE, 2025.
- [57] Shenyuan Gao, William Liang, Kaiyuan Zheng, Ayaan Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, et al. Dreamdojo: A generalist robot world model from large-scale human videos. *arXiv preprint arXiv:2602.06949*, 2026.
- [58] GenRobot AI. RealOmin: 10Kh RealOmin-Open Dataset, 2025. URL <https://huggingface.co/datasets/genrobot2025/10Kh-RealOmin-OpenData>.
- [59] Shihan Wu, Xuecheng Liu, Shaoxuan Xie, Pengwei Wang, Xinghang Li, Bowen Yang, Zhe Li, Kai Zhu, Hongyu Wu, Yiheng Liu, et al. Robocoin: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.
- [60] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, et al. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
- [61] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, YINUO Zhao, Zhiyuan Xu, Guang Yang, et al. Robomind: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.
- [62] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6613–6623, 2024. doi: 10.1109/CVPR52733.2024.00632. URL https://openaccess.thecvf.com/content/CVPR2024/html/Yin_One-step_Diffusion_with_Distribution_Matching_Distillation_CVPR_2024_paper.html.
- [63] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems*, volume 36, pages 49842–49869, 2023. doi: 10.52202/075280-2170. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/9c2aa1e456ea543997f6927295196381-Abstract-Conference.html.
- [64] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Frédo Durand, and William T. Freeman. Improved distribution matching distillation for fast image synthesis. In *Advances in Neural Information Processing Systems*, volume 37, pages 47455–47487, 2024. doi: 10.52202/079017-1505. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/54dcf25318f9de5a7a01f0a4125c541e-Abstract-Conference.html.
- [65] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [66] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [67] Yuyang Zhang, Wen Yao Zhang, Zekun Qi, He Zhang, Haitao Lin, Jingbo Zhang, Yao Mu, Xiaokang Yang, Wenjun Zeng, and Xin Jin. ImageWAM: Do world action models really need video generation, or just image editing? *arXiv preprint arXiv:2606.19531*, 2026.

- [68] Ronghan Chen, Yandan Yang, Zuojin Tang, Dongjie Huo, Tong Lin, Haoning Wu, Haoyun Liu, Yuzhi Chen, Lulu Zheng, Botai Yuan, et al. Abot-m0. 5: Unified mobility-and-manipulation world action model. *arXiv preprint arXiv:2607.00678*, 2026.
- [69] Haoxiang Ma, Junhao Cai, Xiaoxu Xu, Hao Li, Yuyin Yang, Yang Tian, Jiafei Cao, Hongrui Zhu, Zherui Qiu, Yuqiang Yang, et al. Internvla-a1. 5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.
- [70] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems*, 2025.
- [71] Linqing Zhong, Yi Liu, Yifei Wei, Ziyu Xiong, Maoqing Yao, Si Liu, and Guanghui Ren. ACoT-VLA: Action chain-of-thought for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.
- [72] Yifu Xiong, Wenhao Yu, Jiaxuan Lin, Bojun Zou, Jiahao Li, Lu Zhang, Yanyong Zhang, and Jianmin Ji. CAC-VLA: Context-gated action conditioning for vision-language-action models. *arXiv preprint arXiv:2607.04816*, 2026.
- [73] Qiuyue Wang, Mingsheng Li, Jian Guan, Jinhui Ye, Sicheng Xie, Yitao Liu, Junhao Chen, Zhixuan Liang, Jie Zhang, Xintong Hu, et al. Qwen-vla: Unifying vision-language-action modeling across tasks, environments, and robot embodiments. *arXiv preprint arXiv:2605.30280*, 2026.
- [74] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026.
- [75] Jinhui Ye, Ning Gao, Senqiao Yang, Jinliang Zheng, Zixuan Wang, Yuxin Chen, Pengguang Chen, Yilun Chen, Shu Liu, and Jiaya Jia. StarVLA- α : Reducing complexity in vision-language-action systems. *arXiv preprint arXiv:2604.11757*, 2026.
- [76] Haoqi Yuan, Zhixuan Liang, Anzhe Chen, Ye Wang, Haoyang Li, Pei Lin, Yiyang Huang, Zixing Lei, Tong Zhang, Jiazhao Zhang, et al. Qwen-robotmanip technical report: Alignment unlocks scale for robotic manipulation foundation models. *arXiv preprint arXiv:2606.17846*, 2026.
- [77] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, et al. Motus: A unified latent action world model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 35101–35113, 2026.
- [78] Yixiang Chen, Peiyan Li, Yuan Xu, Qisen Ma, Jiabing Yang, Kai Wang, Jianhua Yang, Dong An, He Guan, Gaoteng Liu, et al. Flowwam: Optical flow as a unified action representation for world action models. *arXiv preprint arXiv:2607.13017*, 2026.
- [79] RoboCasa Team. RoboCasa365 Leaderboard. <https://robocasa.ai/leaderboard.html>, 2026. Updated August 21, 2026; accessed August 31, 2026.
- [80] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [81] Dongyoung Kim, Huiwon Jang, Myungkyu Koo, Suhyeok Jang, Taeyoung Kim, et al. Rldx-1 technical report. *arXiv preprint arXiv:2605.03269*, 2026.
- [82] Xiaofeng Wang, Zheng Zhu, Guan Huang, Boyuan Wang, Xinze Chen, and Jiwen Lu. Worlddreamer: Towards general world models for video generation via predicting masked tokens. *arXiv preprint arXiv:2401.09985*, 2024.

A Real-Robot Evaluation Details

This section supplements the aggregate real-robot results in Section 6.1 and Section 6.2 with task-level definitions and evaluation conventions.

A.1 Task Definitions and Success Criteria

All 12 tasks are held out from DROID post-training. Except for the per-object scoring used in the Toys task, a rollout is successful only when the commanded outcome is achieved at the end of the episode.

Standard suite.

- **Stack bowls and place them on a plate.** Both bowls must remain stably stacked on the plate.
- **Pick up the instructed left or right cup.** The cup on the instructed side must be lifted without being knocked over; picking up the cup on the other side is a failure.
- **Move bread from a toaster to a plate.** The bread must be removed from the toaster and left resting on the plate.
- **Stack blocks.** The purple block must remain stably on the instructed green or pink block.
- **Open an air-fryer drawer.** The drawer must be moved from its initial closed state to the open state.
- **Put toys into a box.** Each of the three target toys is scored separately and counts as successful when it is placed inside the blue box.
- **Close a microwave.** The microwave door must be moved from its initial open state to the closed state.
- **Push down a toaster switch.** The toaster switch must reach its activated position.

Visually Perturbed suite.

- **Open an air-fryer drawer.** The drawer must be opened under glare and flashing illumination with surrounding distractor objects.
- **Push down a toaster switch.** The switch must reach its activated position under glare and flashing illumination with surrounding distractor objects.
- **Place two pieces of bread on a plate.** Both pieces of bread must rest on the plate under a tablecloth background and unseen-object clutter.
- **Stack two bowls.** The two bowls must remain stably stacked under a tablecloth background and unseen-object clutter.

A.2 Rollout Protocol and Metrics

All methods use the same robot, camera placement, language instruction, scene reset, action decoder, and safety limits. Each policy receives two external RGB views, one wrist view, and the current proprioceptive state. It predicts a 24-step action chunk and obtains fresh observations before the next closed-loop update. Each task uses 10 rollouts. For seven Standard tasks and all four Perturbed tasks, every rollout has a binary success outcome. The Toys task contains three object placements per rollout; each placement is scored separately, giving 30 binary placement outcomes. We first compute one success rate per task, using 10 rollout outcomes for the other tasks and 30 placement outcomes for Toys. The Standard, Perturbed, and Overall scores are then macro averages over 8, 4, and 12 task-level rates, respectively. Thus, each task has equal weight in its suite despite the finer-grained scoring used for Toys.

A.3 DROID Adaptation Details

Each ZimaBlue variant uses the same two-stage target-robot adaptation. The Slow System is first post-trained on DROID for 60k steps. It is then frozen while the Fast System is trained for 160k steps. Table 8 summarizes the optimization settings shared by the four pre-training initializations.

Table 8: DROID adaptation settings. The Slow System is frozen throughout Fast System training.

Training stage	Steps	Global batch	Learning rate	Warmup	Action horizon
Slow System post-training	60k	256	1×10^{-4}	2%	24
Fast System training	160k	64	1×10^{-4}	1%	24

A.4 Standard Suite Results

The four ZimaBlue variants use, respectively, no cross-embodiment pre-training, multi-embodiment video-action pre-training, video pre-training followed by video-action pre-training, and scaled video pre-training followed by video-action pre-training. All four use the same DROID adaptation described in Appendix A.3. Table 9 reports successful rollouts for every Standard task and evaluation variant.

Table 9: Standard-suite evaluation details. Results for the two baselines and four ZimaBlue variants. Each entry reports successful evaluation units over attempts: rollouts for seven tasks and individual object placements for Toys. Here, VA means multi-embodiment video-action pre-training; V+VA means video pre-training followed by VA; and SV+VA means scaled video pre-training followed by VA. The macro averages correspond to the Standard scores reported in Table 3.

Evaluation details		Released policies		ZimaBlue initialization			
Task	Evaluation setup	$\pi_{0.5}$	Dream Zero	From scratch	VA	V+VA	SV+VA
<i>Stack bowls and place on a plate</i>	pink on blue; blue on pink	5/10	6/10	2/10	2/10	7/10	8/10
<i>Pick up the instructed cup</i>	left cup; right cup	10/10	7/10	3/10	5/10	8/10	10/10
<i>Push down the toaster switch</i>	push down	5/10	6/10	9/10	8/10	9/10	8/10
<i>Close the microwave door</i>	initial door angle 75°	10/10	7/10	0/10	9/10	8/10	9/10
<i>Put all toys into the blue box</i>	random layout; 1000-step horizon	25/30	13/30	19/30	16/30	28/30	28/30
<i>Move bread from the toaster to a plate</i>	left-to-right; right-to-left	7/10	2/10	4/10	4/10	10/10	9/10
<i>Open the air-fryer drawer</i>	pull the drawer open	0/10	10/10	6/10	9/10	6/10	10/10
<i>Stack the purple cube</i>	on pink; on green	7/10	7/10	7/10	4/10	9/10	7/10
Macro average (%)	8 tasks	65.4	61.7	46.7	57.9	82.9	87.9

A.5 Visually Perturbed Suite Results

Table 10 reports results under controlled visual shifts. The bread and bowl tasks combine an unseen scene, a cluttered background, and unseen distractor objects. The appliance tasks introduce dynamic illumination and background clutter; the toaster is also evaluated at a different location.

Table 10: Visually Perturbed-suite evaluation details. Results for $\pi_{0.5}$, DreamZero, and the four ZimaBlue variants. Each entry reports successful rollouts over ten attempts. Here, VA means multi-embodiment video-action pre-training; V+VA means video pre-training followed by VA; and SV+VA means scaled video pre-training followed by VA.

Evaluation details		Released policies		ZimaBlue initialization			
Task	Visual shift	$\pi_{0.5}$	Dream Zero	From scratch	VA	V+VA	SV+VA
<i>Place two pieces of bread on a plate</i>	unseen scene; cluttered background; unseen objects	4/10	0/10	1/10	3/10	5/10	7/10
<i>Stack two bowls</i>	unseen scene; cluttered background; unseen objects	6/10	3/10	1/10	1/10	3/10	5/10
<i>Open the air-fryer drawer</i>	dynamic illumination; background clutter	1/10	7/10	3/10	5/10	3/10	5/10
<i>Push down the toaster switch</i>	dynamic illumination; different location	2/10	5/10	1/10	0/10	3/10	6/10
Macro average (%)	4 tasks	32.5	37.5	15.0	22.5	35.0	57.5

A.6 Perturbation Controls and Failure Analysis

The appliance tasks vary illumination through glare and flashing lights and add nearby objects as visual distractors. The bread and bowl tasks replace the tabletop appearance with a tablecloth and introduce many unseen surrounding objects. The Visually Perturbed score is reported separately from the Standard score to isolate robustness under these controlled shifts. Failure analysis distinguishes instruction or perception errors, grasp and contact failures, accumulated pose error, and failures to recover after partial execution errors.

B Additional Simulation Experiment Details

This appendix provides supplementary experimental details and analyses for the simulation benchmarks evaluated in Section 6.3.

B.1 LIBERO-Plus Detailed Analysis

Evaluation scope and aggregation. We evaluate both protocols on the same 10,030 episodes covering four suites and seven perturbation categories. Category scores pool episodes across suites, and their unweighted mean gives the category-macro result. For Robot Initial States, we follow the official protocol without restoring the perturbed configuration.

Table 11: ZimaBlue performance across the four underlying LIBERO task suites. Values are episode-level success rates (%); Overall is weighted by the number of episodes and therefore differs from the category-macro score in Table 5.

Protocol	Spatial	Object	Goal	LIBERO-10	Overall
Zero-shot	87.8	88.4	81.4	86.5	86.0
SFT	92.8	92.1	87.1	94.0	91.5
Δ	+5.0	+3.7	+5.8	+7.5	+5.5

Table 11 shows that SFT improves all four suites (+3.7 to +7.5 points), with the largest gain on LIBERO-10. Goal remains the hardest suite at 87.1%, so aggregate robustness gains do not eliminate the difficulty of goal-conditioned task execution.

Table 12: Fine-grained effect of LIBERO-Plus supervision. Each entry is the SFT success rate minus the zero-shot success rate in percentage points, computed over the same episodes. Positive values denote gains.

Suite	Camera	Robot	Language	Light	Background	Noise	Layout
Spatial	+42.3	-6.9	-1.8	-1.0	+20.2	-5.4	-9.6
Object	+28.8	-16.8	-3.1	+1.0	+2.4	+7.1	+4.5
Goal	+32.1	-1.5	-5.6	+3.2	+3.6	+7.1	+0.5
LIBERO-10	+45.8	-6.1	0.0	0.0	+4.8	+3.6	-3.2

Table 12 shows consistent camera gains across all suites, largest on LIBERO-10 (+45.8 points), and background gains throughout. Robot Initial States decreases in every suite, most on Object (-16.8 points), while Layout improves on Object and Goal but declines on Spatial and LIBERO-10. These trade-offs explain how SFT raises every suite-level aggregate despite several category-level regressions. Figure 9 shows initial observations for all seven perturbations.

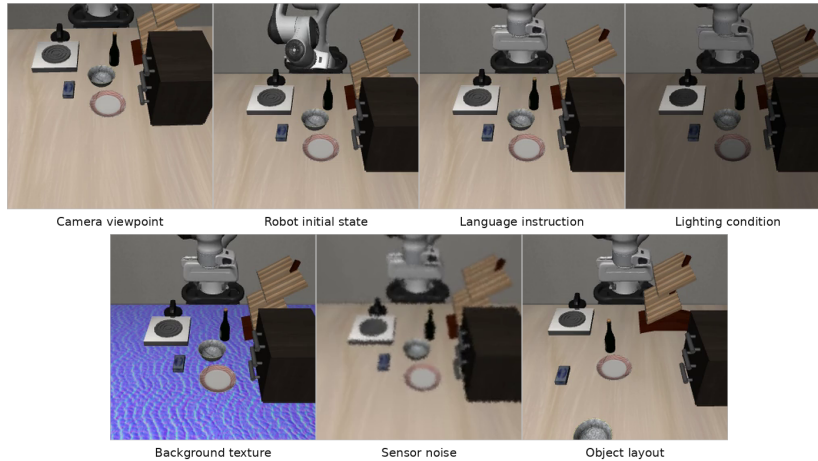


Figure 9: The seven LIBERO-Plus perturbation categories. Initial observations are shown for the same task in our official no-restore SFT evaluation. Language perturbations modify the instruction; quantitative results use all episodes (Table 5).

B.2 RoboTwin 2.0 Per-Task Results

Detailed per-task success rates on RoboTwin 2.0 are reported in Table 13.

Table 13: Per-task success rates (%) on RoboTwin 2.0. Each task is evaluated over 100 episodes in both Clean and Randomized settings; Average is their arithmetic mean.

Task	Clean	Randomized	Average
adjust_bottle	100.0	100.0	100.0
beat_block_hammer	98.0	98.0	98.0
blocks_ranking_rgb	100.0	100.0	100.0
blocks_ranking_size	94.0	92.0	93.0
click_alarmclock	92.0	83.0	87.5
click_bell	100.0	99.0	99.5
dump_bin_bigbin	95.0	92.0	93.5
grab_roller	97.0	97.0	97.0
handover_block	99.0	97.0	98.0
handover_mic	100.0	98.0	99.0
hanging_mug	67.0	80.0	73.5
lift_pot	100.0	99.0	99.5
move_can_pot	100.0	98.0	99.0
move_pillbottle_pad	100.0	99.0	99.5
move_playingcard_away	100.0	97.0	98.5
move_stapler_pad	90.0	83.0	86.5
open_laptop	98.0	100.0	99.0
open_microwave	79.0	87.0	83.0
pick_diverse_bottles	97.0	93.0	95.0
pick_dual_bottles	100.0	100.0	100.0
place_a2b_left	99.0	98.0	98.5
place_a2b_right	95.0	92.0	93.5
place_bread_basket	100.0	99.0	99.5
place_bread_skillet	98.0	95.0	96.5
place_burger_fries	97.0	99.0	98.0
place_can_basket	96.0	91.0	93.5
place_cans_plasticbox	100.0	100.0	100.0
place_container_plate	99.0	99.0	99.0
place_dual_shoes	89.0	79.0	84.0
place_empty_cup	100.0	100.0	100.0
place_fan	100.0	97.0	98.5
place_mouse_pad	56.0	66.0	61.0
place_object_basket	96.0	96.0	96.0
place_object_scale	99.0	92.0	95.5
place_object_stand	98.0	98.0	98.0
place_phone_stand	97.0	99.0	98.0
place_shoe	92.0	98.0	95.0
press_stapler	83.0	85.0	84.0
put_bottles_dustbin	100.0	98.0	99.0
put_object_cabinet	94.0	90.0	92.0
rotate_qrcode	88.0	92.0	90.0
scan_object	95.0	97.0	96.0
shake_bottle	100.0	100.0	100.0
shake_bottle_horizontally	100.0	99.0	99.5
stack_blocks_three	100.0	99.0	99.5
stack_blocks_two	100.0	100.0	100.0
stack_bowls_three	97.0	99.0	98.0
stack_bowls_two	99.0	97.0	98.0
stamp_seal	98.0	99.0	98.5
turn_switch	65.0	71.0	68.0
Overall	94.7	94.3	94.5

B.3 RoboCasa365 Per-Task Results

Detailed per-task success rates on RoboCasa365 are reported in Table 14.

Table 14: Per-task success rates (%) on RoboCasa365. Each task is evaluated over 50 episodes. Average reports the task-level mean within each split, and Overall is averaged across all 50 tasks.

Split	Task	Success
<i>Atomic-Seen</i>	CloseBlenderLid	42.0
	CloseFridge	96.0
	CloseToasterOvenDoor	96.0
	CoffeeSetupMug	44.0
	NavigateKitchen	84.0
	OpenCabinet	92.0
	OpenDrawer	86.0
	OpenStandMixerHead	88.0
	PickPlaceCounterToCabinet	74.0
	PickPlaceCounterToStove	76.0
	PickPlaceDrawerToCounter	88.0
	PickPlaceSinkToCounter	82.0
	PickPlaceToasterToCounter	78.0
	SlideDishwasherRack	88.0
	TurnOffStove	46.0
	TurnOnElectricKettle	90.0
	TurnOnMicrowave	80.0
	TurnOnSinkFaucet	76.0
	Average (%) ↑	78.1
<i>Composite-Seen</i>	DeliverStraw	32.0
	GetToastedBread	28.0
	KettleBoiling	50.0
	LoadDishwasher	54.0
	PackIdenticalLunches	6.0
	PreSoakPan	78.0
	PrepareCoffee	22.0
	RinseSinkBasin	66.0
	ScrubCuttingBoard	90.0
	SearingMeat	16.0
	SetUpCuttingStation	62.0
	StackBowlsCabinet	72.0
	SteamInMicrowave	66.0
	StirVegetables	24.0
	StoreLeftoversInBowl	82.0
	WashLettuce	58.0
	Average (%) ↑	50.4
<i>Composite-Unseen</i>	ArrangeBreadBasket	10.0
	ArrangeTea	2.0
	BreadSelection	36.0
	CategorizeCondiments	10.0
	CuttingToolSelection	46.0
	GarnishPancake	20.0
	GatherTableware	0.0
	HeatKebabSandwich	0.0
	MakeIceLemonade	8.0
	PanTransfer	0.0
	PortionHotDogs	10.0
	RecycleBottlesByType	6.0
	SeparateFreezerRack	14.0
	WaffleReheat	54.0
	WashFruitColander	36.0
	WeighIngredients	12.0
	Average (%) ↑	16.5
Overall Average (%) ↑		49.5

C Attention Masks

Figure 10 summarizes the self-attention dependencies of the Slow and Fast DiTs. We use $i \in \{1, \dots, t\}$ as the temporal index throughout.

Slow DiT. The Slow DiT uses block-causal attention over trajectory segments. Clean context tokens attend only to the current and preceding clean context. For each transition, the noised video and action tokens jointly attend to the preceding clean context, the aligned state, and one another within the same block. They cannot access target tokens from other blocks or the clean outcome of the current or future transitions. This mask enables teacher-forced training of multiple transitions in parallel without leaking future observations. During video-only pre-training, the action and state streams are omitted; the view-validity mask additionally blocks padded camera regions.

Fast DiT. The Fast DiT predicts actions without a future-video query stream. At step i , each action query attends to the updated observation c_i , current state s_i , the complete prefix-conditioned action chunk a_i , and the read-only Slow video K/V cache $\mathcal{K}_{\text{slow}}$. Only action queries fuse these sources; observation and state queries remain local. Because Slow is frozen during Fast training, its cache provides conditioning without receiving gradients.

Attention within the action chunk is non-causal, so every action token can attend to every other valid token in the same request. Different Fast requests remain independent, and each request jointly denoises one action horizon from the latest observation and state.

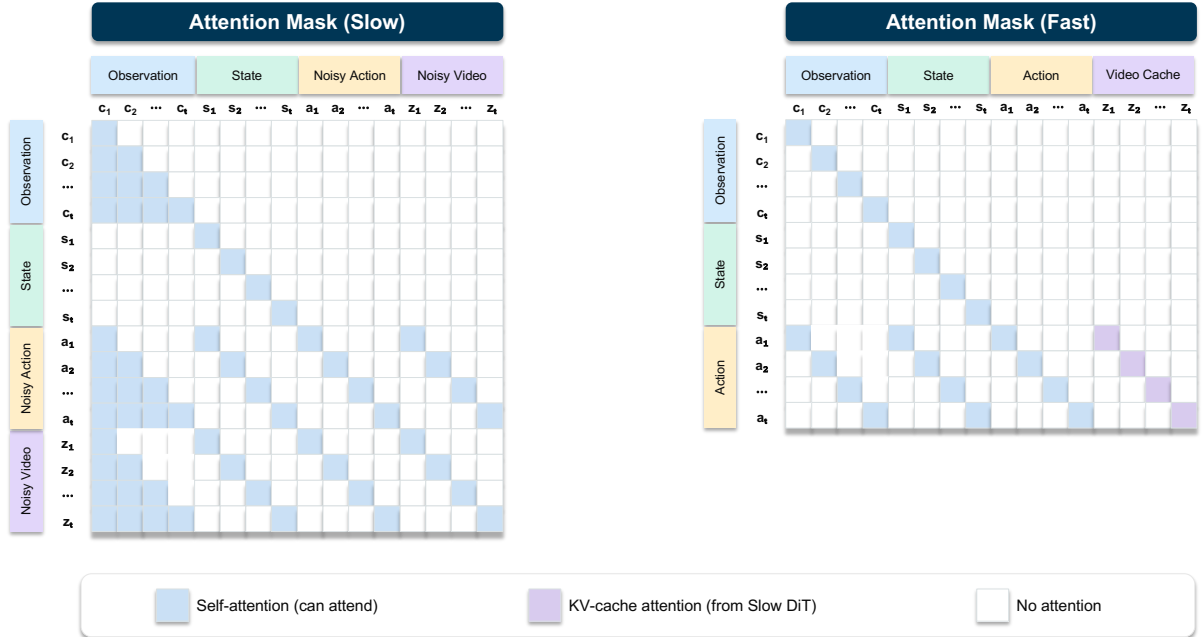


Figure 10: Attention masks of the Slow and Fast DiTs. Blue cells denote K/V blocks produced within the same branch, purple cells denote video K/V features imported from the Slow DiT, and white cells denote blocked dependencies. Rows are queries and columns are keys and values. In the Slow mask, c_i , z_i , s_i , and a_i denote the clean video context, noised future-video latent, proprioceptive state, and noised action chunk at step i . In the Fast mask, c_i , s_i , and a_i denote the updated observation, state, and prefix-conditioned action chunk at step i , while $\mathcal{K}_{\text{slow}}$ is the read-only Slow video K/V cache. Language condition ℓ is supplied through cross-attention and omitted from the self-attention diagram for clarity.